

SYSTEM DESIGN PATTERNS

Senior Architect Edition — First and Final Reference

***How to use this:** Read once fully. Before any interview, read Part 1 and Part 4. For domain-specific prep (real-time, ML systems), read the relevant Part 3 sections. The goal is not memorisation — it's internalising the why so your answers feel like thinking, not recall.*

Quick Navigation

You need...	Go to
How to structure a 45-min interview	Part 1
Numbers and estimation math	Part 2
Pattern catalog (what + when + trade-offs)	Part 3
Quick "which one do I pick" decision tables	Part 4
Estimation worked examples	Part 5
Classic problem skeletons (URL shortener, chat, etc.)	Part 6
Your projects as proof points	Part 7
Full glossary	Part 8

Section map (Part 3 — Core Patterns):

A. Read Scalability · B. Write Scalability · C. Consistency · D. Real-Time · E. Data Storage · F. Caching · G. Messaging · H. API Design · I. Resilience · J. Security · K. Observability · L. Geographic Distribution · M. Storage Patterns · N. Availability & Reliability · O. Architecture Patterns (incl. Hexagonal, DDD, Strangler Fig, Serverless) · P. Infrastructure Fundamentals · Q. Location-Based Patterns · R. Multi-Tenant Architecture · S. Consistent Hashing · T. Bloom Filters · U. Feature Flags · V. Zero-Downtime Schema Migrations · W. Webhooks · X. Connection Pooling

Part 4 — Software Engineering Foundations:

SOLID Principles · GoF Design Patterns (Creational · Structural · Behavioral)

Part 1: The Interview Playbook

The 45-Minute Structure

Misallocating time is the most common failure mode. Candidates spend 30 minutes on requirements, have 10 minutes left, and produce a shallow design.

Phase	Duration	What you're doing
Requirements	5–7 min	Clarify scope, users, scale, constraints
Estimation	3–5 min	Back-of-envelope: QPS, storage, bandwidth
High-level design	8–10 min	Components, data flow, no implementation details yet
Deep dive	15–20 min	Pick 2–3 hard problems and go deep
Trade-offs & evolution	5–7 min	What would break at 10×, what you'd do differently

The cardinal rule: Drive the interview. Don't wait for the interviewer to ask "what about caching?" — bring it up yourself and reason through it.

Clarifying Questions That Always Matter

Before drawing anything:

Scale:

- How many users? DAU vs MAU?
- Read:write ratio?
- Uniform or bursty load? (social feed vs exam deadline burst)
- SLA? (99.9% = 8.7 hrs/year downtime; 99.99% = 52 min/year)

Data:

- What's the data model — relational or document?
- How long do we retain data? Any archival requirements?
- Consistency requirements — bank transfer or social feed?

Scope:

- What's in scope for today? (global availability? analytics? mobile?)
- Any hard constraints? (on-prem, specific cloud, compliance)

The single best question to ask: "What's the hardest problem in this system?" — tells you where the interviewer wants you to go deep.

How to Talk About Trade-offs (The Senior-Level Skill)

Junior engineers pick a tool. Senior engineers explain what they're giving up.

Template for every choice:

"I'll use X because [primary reason]. The trade-off is [what we lose]. That's acceptable here because [why this system can tolerate it]. If that constraint changes, we'd revisit by [alternative]."

Examples:

- "I'll use eventual consistency here because a user seeing a feed post 200ms late is acceptable. If this were a bank balance, I'd need linearisability and strong reads from the primary."
- "I'll use SQS FIFO over Standard because order matters within an exam session. The trade-off is 3,000 msg/sec per group vs unlimited — but per-exam traffic is well under that."

- "I'll use Redis for this cache and accept data loss on restart. If durability is required, we'd enable AOF persistence — but that trades write latency."

What Interviewers Actually Evaluate

Dimension	What they watch for
Breadth	Do you know the tool landscape? Can you name the right solution category?
Depth	Do you understand <i>why</i> a tool works? Can you explain its internals?
Trade-offs	Do you acknowledge what you're giving up? Can you defend your choices?
Communication	Do you think out loud? Is your answer structured?
Failure thinking	Do you ask "what breaks?" not just "how does it work?"
Numbers	Can you sanity-check your design with rough math?
Pragmatism	Do you know when the simple solution is right?

Handling Pushback

If the interviewer says "that won't scale" or "that's not right":

1. **Don't capitulate immediately.** "Can you help me understand the concern you see?"
2. **Acknowledge the valid point.** "You're right that X becomes a problem at N scale."
3. **Offer the evolution.** "At current scale I'd start simple; as we cross Y threshold, I'd migrate to Z."

Interviewers often test whether you'll fold or hold.

Part 2: Numbers You Must Know Cold

Latency Hierarchy

Operation	Latency
CPU L1 cache hit	~0.5 ns
L2 cache hit	~7 ns
RAM read	~100 ns
SSD read (sequential)	~100 µs
HDD seek	~10 ms
Same-datacenter round-trip	~0.5 ms
Cross-region (US East → West)	~40 ms
US → Europe	~80 ms
US → Asia Pacific	~150 ms

Two numbers that matter most: RAM is ~100× faster than SSD. SSD is ~100× faster than HDD.

Throughput Benchmarks (rough, for estimation)

System	Read QPS	Write QPS
Postgres (single node, tuned)	~10,000	~2,000
Postgres + read replicas	~50,000+	~2,000
Redis (single node)	~100,000	~100,000
Cassandra (cluster)	~500,000	~500,000
Kafka (per partition)	~1M msgs/sec	~500K msgs/sec
Typical REST API (Django/FastAPI)	~2,000–5,000	~1,000–2,000

Storage Quick Reference

Data type	Typical size
Tweet / short text	~280 bytes
User profile row	~1 KB
Photo (compressed)	~200 KB
HD video (1 min)	~150 MB
4K video (1 min)	~400 MB

The Estimation Framework (10 steps)

- 1. Ask: DAU (daily active users)
- 2. Ask: reads vs writes ratio
- 3. Compute: QPS = DAU × actions/day ÷ 86,400
- 4. Add: peak multiplier (2–5×)
- 5. Compute: storage/year = write QPS × data size × 86,400 × 365
- 6. Compute: bandwidth = read QPS × response size
- 7. Ask: replication factor (typically 3×)
- 8. Apply: cache hit ratio (80% cache hit = 20% of reads reach DB)
- 9. Round aggressively: 86,400 ≈ 10⁵
- 10. **State assumptions out loud** — interviewers credit the reasoning, not just the number

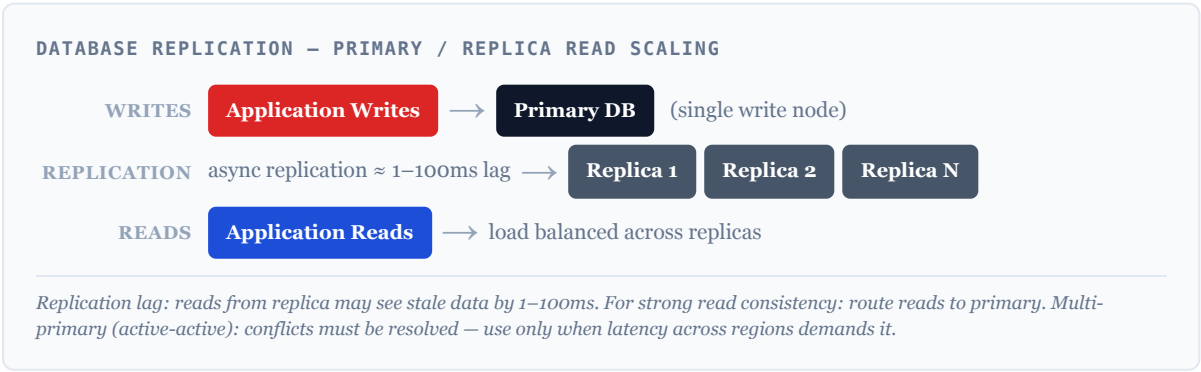
Mental Math Shortcuts

Fact	Value
Seconds in a day	86,400 ≈ 10 ⁵
1M DAU × 1 action/day	~12 QPS
1B DAU × 1 action/day	~12,000 QPS
1M photos/day × 200KB	200 GB/day storage
Twitter at peak	~500K reads/sec, ~6K writes/sec

Part 3: Core Patterns Catalog

Each entry: **What it is** → **Use when** → **Avoid when** → **Key trade-off** → **Failure mode** → **Proof point**

A. Read Scalability Patterns



A1. Read Replicas

What it is: Secondary copies of your primary database that serve read queries while the primary handles writes only.

Use when: Reads >> writes. Queries can tolerate slight staleness (replication lag). You've already exhausted indexes and connection pooling.

Avoid when: You need read-your-own-writes consistency. Replication lag is unacceptable (financial balances, inventory).

Key trade-off: Replication lag (typically 1ms–1s, can spike under load). Reads may return stale data.

Failure mode: If replica lag grows during a primary overload event, replicas compound the problem — clients get stale data even after writes succeed.

Interview move: "Before adding replicas, I'd check: are we connection-pool-limited (PgBouncer), query-limited (missing index), or truly throughput-limited? Replicas only solve the last one."

Master-Slave vs Master-Master Replication

	Master-Slave	Master-Master
Writes	Primary only	Both nodes accept writes
Reads	Primary + replicas	Both nodes
Conflict risk	None	Yes — concurrent writes to same row need resolution
Failover	Promote replica (manual or automatic)	Other master continues immediately
Use when	Read scaling; simple consistency	High write availability; geographic distribution

Master-Master conflict resolution: Last-write-wins (LWW) by timestamp is simplest but risks losing data on clock skew. Application-level merge is safest but complex. Most systems avoid master-master for transactional data and use it only where conflicts are rare or acceptable (e.g., session stores, counters).

A2. CDN (Content Delivery Network)

What it is: Geographically distributed proxy servers that cache and serve content from the edge nearest the user.

Use when: Static assets (JS, CSS, images, video). Read-heavy content with infrequent updates. Global user base needing low latency.

Avoid when: Content is personalised per user. Content updates need instant global propagation.

Key trade-off: Cache invalidation is hard. CDN serves stale content until TTL expires or you actively purge.

Fix for stale content on deploy: Content-hash filenames (`main.abc123.js`). The URL changes on every deploy, forcing a cache miss globally.

Push CDN vs Pull CDN — The Fundamental Decision

This is the first architectural choice you make when introducing a CDN. Everything else (TTL, invalidation) is secondary.

Dimension	Pull CDN	Push CDN
How it works	Content stays on your origin. First request → CDN pulls from origin → caches at edge. Subsequent requests → cache hit.	You upload content to CDN edge nodes yourself, before any user requests it.
First request	Slow (cache miss → origin pull)	Fast (content already at edge)
Who manages the edge	CDN manages itself — caches on demand, evicts on TTL	You manage it — you push updates, you control what's cached
Storage cost	Only caches what's been requested	Pays for everything you push, regardless of demand
Best for	Unpredictable traffic, large libraries, content you can't pre-predict	Predictable spikes, small known content set, large media files
Stale content risk	TTL-based; old content served until expiry	You control it — push the new version when it's ready
Example	A blog's images — cached on first visitor, served from edge after	Netflix pre-positioning episode files to ISP-level edges hours before a premiere

When Push CDN wins:

- You know the traffic spike is coming (movie premiere at 8pm — push files at 7pm)
- The content is large and an origin pull during a spike would overwhelm your servers
- You have a small, predictable content set (a software release, a sporting event's media pack)

When Pull CDN wins:

- Content library is large and you can't predict what will be popular
- Traffic patterns are organic and unpredictable
- You want zero operational overhead — the CDN manages itself

The hybrid reality: Most large platforms use both. Static assets (JS, CSS) use pull. Large known media files (a film releasing at midnight) use push. The decision is per content type, not per system.

Interview move: "Before picking pull or push, I'd ask: do we know in advance what content will be popular, and can we predict when? If yes, push gives us a guaranteed cache hit from the first request. If the content library is large and access is unpredictable, pull with a good TTL strategy is operationally simpler."

A3. Denormalisation

What it is: Storing pre-computed or duplicated data to speed up reads, at the cost of write complexity.

Use when: Joins are the read bottleneck. Data is read far more than written. Data rarely changes.

Avoid when: Data changes frequently — you'll have consistency nightmares maintaining multiple copies.

Key trade-off: Every write must update multiple places. Consistency is maintained in application code, not the DB.

A4. Search Index (Elasticsearch / OpenSearch)

What it is: A separate index built from your primary store, optimised for full-text search, faceting, and aggregation.

Use when: Full-text search (LIKE '%query%' on Postgres is a full table scan). Faceted filtering. Autocomplete, fuzzy matching, synonyms.

Avoid when: You need transactional consistency between index and DB. The query could be served by a proper Postgres index.

Key trade-off: The index is always slightly behind the primary. You need a sync mechanism (CDC, dual-write, or change stream).

Failure mode: Index drift — DB has the record, search doesn't. Fix: periodic full reindex + real-time CDC for ongoing sync.

A5. Load Balancing

What it is: Distributes incoming requests across multiple servers so no single server becomes the bottleneck.

Layer 4 vs Layer 7

	Layer 4 (Transport)	Layer 7 (Application)
Operates on	TCP/UDP packets	HTTP requests
Sees	IP + port	URL, headers, cookies, body
Speed	Faster (no parsing)	Slower (full HTTP parsing)
Routing logic	IP hash, round-robin	Path-based, header-based, host-based
Use when	Raw throughput matters, non-HTTP traffic	You need content-aware routing

Load Balancing Algorithms

Algorithm	How	Use when
Round-robin	Requests go to each server in turn	Servers are identical, requests are similar weight
Least connections	Route to server with fewest active connections	Requests have variable duration (some are slow)
IP hash	Hash the client IP → always same server	Session affinity needed without sticky cookies
Weighted round-robin	Servers get requests proportional to their weight	Servers have different capacities
Consistent hashing	Hash the request key → same server for same key	Cache locality (CDN, distributed cache)

Sticky Sessions (Session Affinity)

Forces a client to always hit the same server. Required when in-memory session state lives on the server (not in a shared store).

The problem it creates: One server becomes a hot spot. If it dies, that user's session is gone.

The better solution: Store session state externally (Redis, DB). Then any server can handle any request — no stickiness needed.

Health Checks

The load balancer must know which servers are healthy before routing to them.

- **Active health check:** LB pings `/health` on a schedule. If N consecutive checks fail, the server is removed from rotation.
- **Passive health check:** LB observes real traffic. If a server returns too many 5xx errors, it's removed.

Liveness vs Readiness (critical for Kubernetes):

- **Liveness probe:** Is the process alive? If not → restart the container.
- **Readiness probe:** Is the server ready to receive traffic? If not → remove from load balancer pool (but don't restart). Used during startup, DB migrations, graceful shutdown.

Interview move: "I'd always store session state in Redis, not in-process. That makes servers stateless, eliminates sticky session headaches, and lets me add or remove instances freely."

A6. Horizontal vs Vertical Scaling

Vertical scaling (scale up): Buy a bigger machine. More CPU cores, more RAM, faster SSD.

- Simple — no code changes, no distributed systems complexity
- Hard ceiling — there's a maximum machine size
- Single point of failure — one big machine is one failure domain
- Cost-inefficient at the top end

Horizontal scaling (scale out): Add more machines. Run multiple instances behind a load balancer.

- No hard ceiling — add machines as needed

- Requires stateless services (sessions in Redis, not in-process)
- Requires a load balancer
- More complex to operate

The escalation path for any system bottleneck:

1. Profile first — find the actual bottleneck (CPU, memory, I/O, network)
2. Optimise the code or query first — often the real fix
3. Vertical scale — simple, fast, no architecture change
4. Horizontal scale — when vertical ceiling is hit or HA is required

Interview move: "I'd always ask what the bottleneck actually is before scaling. Horizontal scaling solves throughput. Vertical scaling solves latency (bigger machine, faster single-request execution). They're not interchangeable."

A7. Forward Proxy vs Reverse Proxy

Two completely different things that share the word "proxy." Confusing them in an interview is a red flag.

	Forward Proxy	Reverse Proxy
Sits in front of	Clients	Servers
Who configures it	The client (or their org)	The server operator
Client knows target server?	No — proxy forwards on client's behalf	Yes — client calls the proxy; proxy picks backend
Server knows real client?	No — sees proxy IP	No — sees proxy IP (unless X-Forwarded-For header)
Primary use cases	Corporate internet filtering, anonymity (VPN-like), caching for internal users	Load balancing, SSL termination, caching, WAF, DDoS protection
Examples	Squid, corporate VPNs	NGINX, HAProxy, AWS ALB, Cloudflare

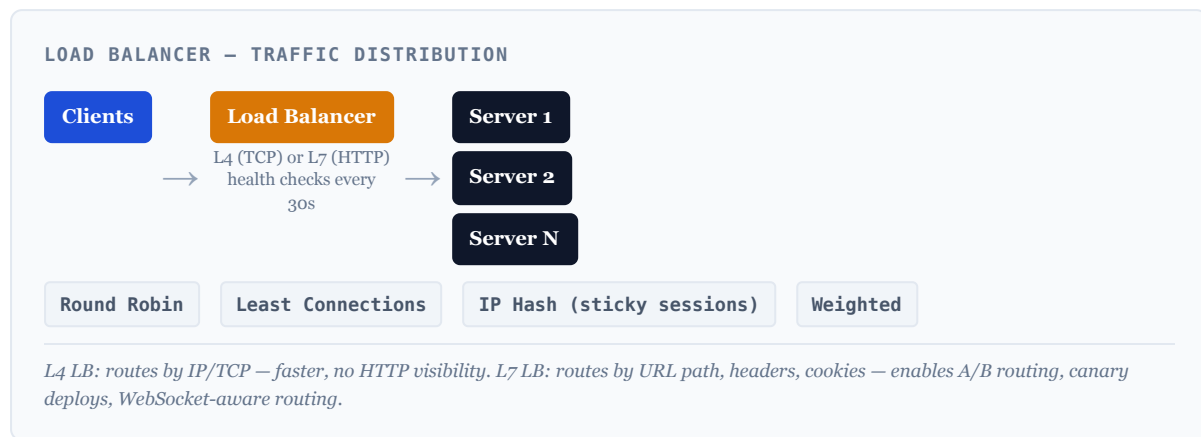
Forward proxy — the corporate firewall model: Every employee's browser is configured to route through the proxy. The proxy can block YouTube, cache popular sites, and log all traffic. The external website sees the proxy's IP, not the employee's.

Reverse proxy — the standard web architecture model: Users call `api.example.com`. That resolves to a reverse proxy (NGINX). The proxy terminates SSL, inspects the request, and forwards to the appropriate backend service. The backend sees `127.0.0.1` or the X-Forwarded-For header.

Why reverse proxy in every production system:

- **SSL termination** — handle TLS once at the edge; backend services speak plain HTTP internally
- **Load balancing** — distribute across multiple backend instances
- **Caching** — cache static responses; backends never touched for cached paths
- **WAF / DDoS** — absorb and filter malicious traffic before it reaches the application

B. Write Scalability Patterns



B1. Message Queue / Async Processing

What it is: Decouple the request (accepted immediately) from the work (processed later). Producer → Queue → Consumer.

Use when: Work takes longer than an acceptable HTTP response. Work is idempotent and retryable. Load is bursty (queue absorbs spikes).

Avoid when: The client needs a synchronous result. Work must happen in strict global order across all producers.

Key trade-off: Client gets "Accepted", not a result. You must build a status-check mechanism (polling or callback).

Failure mode: Consumers fall behind. Queue depth grows unboundedly. Fix: consumer autoscaling on queue depth + DLQ for poison messages.

Proof point: CodeMas – POST /submit → 201 → SQS FIFO → Lambda → result in Postgres → client polls GET /result/{id}/ every 1.5s.

B2. CQRS (Command Query Responsibility Segregation)

What it is: Separate your write model (commands) from your read model (queries). Different stores or schemas optimised for each.

Use when: Read and write patterns are fundamentally different. Read scalability needs vastly exceed write needs.

Avoid when: System is simple. CQRS adds operational complexity that plain CRUD doesn't need.

Key trade-off: Eventual consistency between write and read models. Synchronisation complexity.

Interview move: "I'd suggest CQRS only after hitting the ceiling on a single store. Start simple, extract read models when you have a specific scaling problem."

B3. Event Sourcing

What it is: Store the full sequence of events that led to current state, not current state itself. State = replay of all events.

Use when: You need a complete audit trail (financial, compliance). You need to replay history or time-travel to any past state. Multiple downstream consumers need different projections.

Avoid when: Your team isn't experienced with it — it's a significant mental model shift. Querying current state frequently (requires projections + snapshots).

Key trade-off: Current-state queries require building and maintaining projections (read models). Schema evolution of historical events is painful.

B4. Database Sharding

What it is: Horizontally partition your database by a shard key. Each shard holds a subset of rows on a separate server.

Use when: Single-node write throughput is saturated. Data volume exceeds one machine's capacity.

Avoid when: You haven't exhausted vertical scaling, connection pooling, and read replicas first. Queries require cross-shard joins.

Key trade-off: Cross-shard queries become expensive. Resharding is painful. Uneven shard keys create hot shards.

Good shard key properties:

1. High cardinality (many possible values)
2. Evenly distributed (no hot keys)
3. Co-locates related data (all of one user's data on the same shard)
4. Immutable (user_id = good; user_location = bad)

B5. Rate Limiting

What it is: Control the rate at which a client can make requests. Rejects (HTTP 429) or queues requests exceeding the limit.

Algorithm	How it works	Use when
Token bucket	Bucket holds N tokens; refills at R/sec; each request consumes 1	Allow burst up to N, steady-state at R. Most common.
Leaky bucket	Requests queue; processed at fixed rate	Smooth output regardless of input bursts
Fixed window counter	Count requests per window; reset at boundary	Simple; has burst problem at window edges
Sliding window log	Log each request timestamp; count within rolling window	Accurate; memory-intensive
Sliding window counter	Weighted average of current + previous window	Good balance of accuracy and memory

Storage: Redis — atomic INCR + TTL. Distributed token bucket uses Lua scripts for atomicity.

C. Consistency & Coordination

C1. CAP Theorem

A distributed system can guarantee at most two of: **C**onsistency, **A**vailability, **P**artition Tolerance. Since partitions *will* happen, the real choice is CP vs AP.

Choice	Behaviour during partition	Examples
CP	Returns error rather than stale data	Postgres, HBase, Zookeeper
AP	Returns potentially stale data rather than error	Cassandra, DynamoDB, CouchDB

Interview move: "CAP is the partition scenario. PACELC is the everyday question — even without a partition, we trade latency vs consistency. That's usually more relevant to product decisions."

C2. Consistency Levels (weakest to strongest)

Level	Guarantee	Use case
Eventual	All nodes will eventually agree	Social feeds, DNS, metrics
Monotonic read	A client never reads older data than it has already read	Session reads
Read-your-own-writes	A client always sees its own writes	Post a comment, see it immediately
Causal	Causally related operations appear in order	Comment threads
Strong (linearisable)	Every read returns the latest write from any client	Bank balances, inventory

Proof point: CodeMas uses eventual — client polling every 1.5s means seeing a result 3 seconds late is acceptable. A bank balance is not acceptable with eventual consistency.

C3. Distributed Locks

What it is: Mutual exclusion across processes or machines. Only one holder at a time.

Implementation	Properties	Use when
Redis SETNX + TTL	Simple, widely used	Most use cases; accept TTL risk
Redlock (multi-node Redis)	More robust	Require stronger guarantees
Zookeeper ephemeral nodes	Strong consistency; leader election standard	Critical section, leader election
Postgres advisory locks	<code>pg_try_advisory_lock(key)</code>	DB-centric systems; no extra infra

TTL Goldilocks problem: Too short — holder's work outlasts the lock (another holder starts). Too long — failures leave the lock held indefinitely.

Proof point: CodeMas — `SELECT FOR UPDATE` on `attempt_counter` is a row-level Postgres lock. Prevents two concurrent submissions both passing the attempt limit check.

C4. Saga Pattern

What it is: Implement a distributed transaction as a sequence of local transactions. On failure, compensating transactions roll back completed steps.

Style	How	Pros	Cons
Choreography	Each service reacts to events	Decentralised	Hard to track overall state
Orchestration	Central saga orchestrator commands each service	Easy to debug	Single point of failure

Use when: Multi-service transactions without 2PC. Steps can be compensated on failure (refund, cancel reservation).

Avoid when: Compensation is impossible. You need true atomicity.

C5. Leader Election

What it is: Ensure exactly one node acts as leader for a responsibility at any time. All others are followers.

Algorithms:

- **Raft** — understandable consensus; used in etcd, Consul, CockroachDB
- **Paxos** — original consensus; complex to implement correctly
- **Bully** — node with highest ID wins; simple but not partition-safe

Use when: Single writer needed (primary DB, cron scheduler). Anti-split-brain: two nodes must never both believe they're leader.

C6. Two-Phase Commit (2PC)

What it is: A protocol for achieving an atomic transaction across multiple independent databases or services. Either all participants commit, or all abort.

Two phases:

1. **Prepare phase:** The coordinator asks every participant "can you commit?" Each participant writes the transaction to its WAL and replies Yes or No.
2. **Commit phase:** If all say Yes, coordinator sends Commit to all. If any say No, coordinator sends Abort to all.

The fatal problem — coordinator failure: If the coordinator crashes after sending Prepare but before sending Commit, participants are stuck in a locked state waiting indefinitely. This is the

"blocking" nature of 2PC.

Property	2PC	Saga
Atomicity	True atomicity — all or nothing	Eventual — compensating transactions on failure
Blocking on failure	Yes — participants lock if coordinator dies	No — each step independent
Performance	Slow — multiple round trips, locks held	Fast — local commits, async
Use when	Short, fast transactions requiring true ACID	Long-running, multi-service workflows

Why Saga beats 2PC for microservices: In a microservices world, locking resources across services for the duration of a coordinator round-trip is too expensive and too fragile. Saga accepts eventual consistency in exchange for availability.

When 2PC is still used: Within a single database cluster where the coordinator is the DB itself (Postgres distributed transactions). Not across independent services.

D. Real-Time & Streaming

REAL-TIME DELIVERY — THREE PATTERNS COMPARED

Client Polling

Repeats every N seconds. Simple, stateless server. Latency = poll interval. Use when: simplicity matters, result may not arrive for seconds.

SSE (Server-Sent Events)

One persistent HTTP connection, server pushes. Browser auto-reconnects. Works through proxies. Use when: server streams, client only listens (LLM token streaming).

WebSocket

Bidirectional full-duplex. HTTP upgrade handshake. Use when: client must send during stream — collaborative edit, live cursor, stop-generation button.

D1. Comparison Table

Mechanism	Direction	Connection	Reconnect	Scale	Use when
Client polling	Client → Server	New HTTP each poll	Automatic	Easy (stateless)	Result in DB after async process
Long polling	Server → Client	Held open until event	Manual	Medium	Low-frequency events, HTTP only
SSE	Server → Client	Persistent HTTP stream	Auto (EventSource)	Medium	Dashboard, notifications, 1-way push
WebSocket	Bidirectional	Persistent TCP	Manual	Harder (stateful)	Chat, live collaboration, gaming
Kafka	Server → Consumer groups	Persistent, offset-based	From offset	High + durable	Replayable streams, multi-consumer

D2. When Each Wins

Client polling: Result is written to DB after async work (Lambda writes to Postgres → client polls). Stateless, easy to scale horizontally.

SSE: Server has an event to push. One-directional. Simpler than WebSocket. Note: requires held HTTP connections — each client occupies a server thread unless using async (gevent, asyncio).

WebSocket: Both sides need to send. Latency < 100ms required. Note: stateful — needs sticky sessions or a pub/sub layer (Redis) behind a load balancer.

Kafka: Durable, replayable, multiple independent consumers at massive scale. Use when you can't afford to lose messages and need replay.

D3. CDC (Change Data Capture)

What it is: Reading the database's write-ahead log (WAL) to capture every insert/update/delete as an event, without changing application code.

Tools: Debezium (open source), AWS DMS, PG Logical

Use when: Syncing data to another store (Elasticsearch, data warehouse) without touching application code. Invalidating caches on DB change. Getting events from legacy code that doesn't publish events.

Key trade-off: WAL consumers must keep pace with write rate, or lag grows unboundedly.

E. Data Storage Selection

E1. The Decision Framework (ask in order)

1. What access pattern dominates?

- Key lookups → Redis or DynamoDB
- Relational queries with joins → Postgres
- Full-text search → Elasticsearch
- Ordered time-series → Cassandra or TimescaleDB
- Graph traversal → Neo4j
- Semantic similarity → pgvector or Pinecone

2. What consistency is required?

- Strong ACID → SQL (Postgres, MySQL)
- Eventual ok, scale matters → NoSQL (Cassandra, DynamoDB, MongoDB)

3. What's the write pattern?

- High write throughput, append-only → Cassandra (LSM tree)
- Moderate writes with complex queries → Postgres (B-tree)

4. What's the scale?

- < 10M rows, < 10K QPS → Postgres with good indexes
 - > 100M rows, > 50K read QPS → Read replicas + cache
 - > 1B rows or > 100K write QPS → Sharding or Cassandra
-

E2. Database Quick Reference

DB	Type	Sweet spot	Avoid when
PostgreSQL	Relational	Complex queries, ACID, < 10TB	Write throughput > 10K/s without sharding
Redis	Key-value + data structures	Cache, session, leaderboard, pub/sub, rate limit	Primary datastore (memory-only risk)
MongoDB	Document	Flexible schema, nested docs, heterogeneous data	Highly relational data requiring joins
Cassandra	Wide-column	Time-series, IoT, write-heavy, high availability	Complex queries, joins, aggregations
DynamoDB	Key-value / document	Serverless, auto-scale, AWS-native, simple access	Complex queries, no fixed access pattern
Elasticsearch	Search	Full-text, faceted filtering, analytics	Primary transactional store
ClickHouse	Columnar OLAP	Analytics, aggregations over large datasets	Frequent single-row updates
TimescaleDB	Time-series (Postgres ext.)	Metrics, IoT sensor data, monitoring	General relational queries
Neo4j	Graph	Social graphs, recommendations, fraud detection	Non-graph data
pgvector	Vector (Postgres ext.)	Semantic search, RAG, < 100M vectors	Billions of vectors
Pinecone	Vector (managed)	Billion-scale semantic search	Self-hosted or cost-sensitive

E3. Database Federation

What it is: Split a single monolithic database into multiple databases by functional domain. Users DB, Products DB, Orders DB — each an independent database server.

Different from sharding: **sharding splits rows of the same table** across servers. **Federation splits tables by function** — different concerns live on different servers entirely.

	Sharding	Federation
Splits by	Row (horizontal)	Function/domain (vertical)
Example	Users 0-499K on shard 1, 500K-1M on shard 2	User DB, Product DB, Order DB
Cross-partition joins	Hard (cross-shard)	Hard (cross-DB)
Scales	Write throughput for one table	Independent scaling per domain
Use when	One table is too big for one machine	Different tables have different access patterns

When federation wins: The User service reads users all day. The Order service reads orders all day. If they share a DB, they share connection pools, query plans, and failure domains. Federation gives each service its own DB, letting them scale independently. This is a common step on the microservices journey.

The downside: No cross-DB joins. If the Order service needs user details, it either calls the User service API or accepts some data duplication.

E4. Materialized Views

What it is: A pre-computed query result stored as a physical table. Unlike a regular view (which re-runs the query on every access), a materialized view is computed once and stored — reads hit the stored result.

Use when: A query is expensive (joins, aggregations) and the underlying data doesn't change every second. Dashboard metrics, reporting tables, pre-computed leaderboards.

```
CREATE MATERIALIZED VIEW course_stats AS
  SELECT course_id, COUNT(*) as total_students, AVG(score) as avg_score
  FROM enrollments
  GROUP BY course_id;

REFRESH MATERIALIZED VIEW course_stats; -- run on schedule or on data change
```

Trade-off: Staleness. The view reflects data at the last refresh. Refresh frequency = freshness vs compute cost. `CONCURRENTLY` flag lets Postgres refresh without locking reads.

Interview move: "Instead of adding an index or denormalising, I'd first ask: can this query be a materialized view refreshed every 5 minutes? It gives read performance without touching the write path."

E5. The N+1 Query Problem

What it is: A loop that fires one query to get N parent records, then fires N additional queries to fetch related records for each parent — N+1 queries total.

```
# N+1: fetches 1 course list, then 1 query per course for instructor
courses = Course.objects.all()           # 1 query
for course in courses:
    print(course.instructor.name)         # N queries - one per course
```

```
# Fixed: one query with JOIN
courses = Course.objects.select_related('instructor').all() # 1 query
```

Why it matters at scale: 100 courses = 101 queries. 10,000 courses = 10,001 queries. Each query has network round-trip overhead. At 10ms per query, 10,001 queries = 100 seconds. One JOIN =

10ms.

Fix strategies:

- **Eager loading** — `select_related()` (Django), `include()` (Rails), `JOIN FETCH` (JPQL)
- **Batch loading** — fetch all IDs first, then `WHERE id IN (...)` — one query for all children
- **DataLoader pattern** — batches and deduplicates queries in GraphQL resolvers

F. Caching Strategies



F1. Strategy Comparison

Strategy	Read flow	Write flow	Staleness risk	Use when
Cache-aside (lazy)	Miss → app reads DB → app fills cache	App writes DB; cache invalidated or TTL expires	Medium	Most common; app controls cache
Write-through	Hit → serve cache	App writes cache AND DB simultaneously	Low	Must be fresh immediately after writes
Write-behind	Hit → serve cache	App writes cache only; async flush to DB	Very low reads; data loss risk on crash	Extreme write throughput
Read-through	Miss → cache reads DB automatically	App writes DB; cache updated on next read	Medium	ORM-style caching
Refresh-ahead	Always hit (proactively refreshed)	Cache refreshes entries automatically before TTL expires, based on predicted access	Very low	Predictable access patterns (news homepage, trending content)

F2. Cache Invalidation Strategies

Strategy	How	Use when
TTL	Set expiry; accept staleness window	Data changes infrequently
Event-driven	Invalidate on write event	Data changes frequently; must be fresh
Stampede protection	Lock + background refresh	High traffic; many concurrent misses on same key
Content-hash key	URL includes hash of content (assets)	Static assets; never invalidate
Stale-while-revalidate	Serve stale; refresh in background	UI needs speed; slightly stale is ok

F3. Common Cache Problems

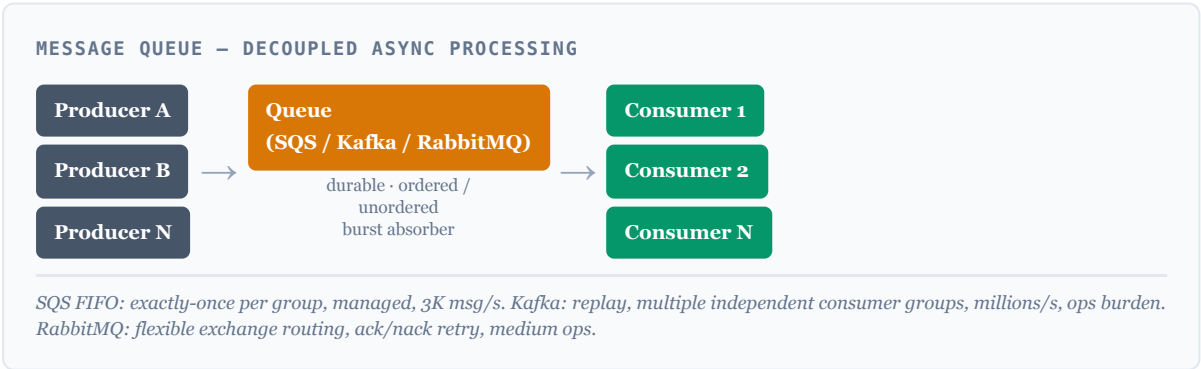
Cache stampede (thundering herd): Popular key expires. Thousands of concurrent requests all miss and hammer the database simultaneously.

- Fix: Probabilistic early expiration (refresh slightly before TTL) or fetch-lock (only one thread fetches; others wait).

Hot key: One key (viral tweet, popular product) gets millions of reads/sec. Single Redis node becomes the bottleneck.

- Fix: Local in-process LRU cache in the app, or key sharding (prefix with random suffix, round-robin reads).

G. Messaging & Event Patterns



G1. Queue vs Pub/Sub vs Streaming

Pattern	Delivery	Consumers	Replay?	Use when
Queue (SQS, RabbitMQ)	One consumer per message	Competing consumers	No	Background jobs; work queue
Pub/Sub (SNS, Redis pub/sub)	All subscribers	Fan-out	No	Notifications; event fan-out
Streaming (Kafka, Kinesis)	One consumer group	Multiple independent groups	Yes (from offset)	Audit log; replayable events; multi-consumer

G2. Delivery Guarantees

Guarantee	Meaning	Cost	Use when
At-most-once	May be dropped; never duplicated	Lowest	Metrics, telemetry — loss is ok
At-least-once	Delivered at least once; may duplicate	Medium	Default; make consumers idempotent
Exactly-once	Exactly once, guaranteed	Highest	Financial transactions, inventory

Pragmatic rule: Design for at-least-once + idempotent consumers. Exactly-once is expensive and often unnecessary.

Idempotency key pattern: Each message carries a unique ID. Consumer checks: "Have I processed this ID?" (Redis SET NX or DB upsert). If yes, skip.

G3. The Outbox Pattern

Problem: You write to DB and publish to a queue. If the publish fails after the DB write, you have inconsistency.

Solution: Write the event to an `outbox` table in the same DB transaction as your domain write. A separate process reads the outbox and publishes to the queue.

```
BEGIN;
  INSERT INTO orders (id, ...) VALUES (...);
  INSERT INTO outbox (event_type, payload) VALUES ('order_created', '{...}');
COMMIT;
-- Outbox poller reads committed rows and publishes to queue
```

The event is published if and only if the DB transaction committed.

G4. Backpressure

What it is: A mechanism for a consumer to signal to a producer to slow down when it can't keep up. Without backpressure, a fast producer overwhelms a slow consumer until memory is exhausted or messages are dropped.

The problem without backpressure:

- Producer writes 50K messages/sec into a queue
- Consumer processes 10K messages/sec
- Queue depth grows 40K/sec → eventually exhausts memory or hits max depth → messages dropped or OOM crash

How to handle it:

Approach	How	Use when
Bounded queue	Queue has a max size; producer blocks or drops when full	You control both producer and consumer
Consumer autoscaling	Add consumers when queue depth grows past threshold	Cloud environment, elastic workloads
Rate limiting the producer	Producer slows down when queue depth is high	Producer can tolerate being throttled
Load shedding	Deliberately drop low-priority messages when overloaded	Real-time systems where stale data is useless
Reactive Streams / async back-pressure	Protocol-level signalling (gRPC flow control, Kafka consumer lag)	Library/framework handles it automatically

Interview move: "I'd instrument queue depth and consumer lag as key metrics. Autoscale consumers when depth exceeds a threshold. Add a DLQ for messages that fail after max retries. If the queue is chronically deep, the root cause is either an underprovisioned consumer fleet or a burst the system was never designed for."

G5. Schema Evolution (Kafka / Event Streams)

The problem: You have 100 consumers reading a Kafka topic. You need to change the message format (add a field, rename a field, change a type). How do you do it without breaking existing consumers?

Rules for safe evolution:

1. **Always add fields as optional with defaults** — existing consumers that don't know the field skip it; new consumers can use it
2. **Never rename a field** — to consumers it looks like deletion + addition; use an alias first
3. **Never change a field's type** — int → string breaks deserialization
4. **Never remove a required field** — old producers still send it; new consumers break

Schema Registry: A central service (Confluent Schema Registry, AWS Glue) that stores and versions Avro/Protobuf schemas. Producers register a schema; consumers validate against it. The registry enforces compatibility rules (backward, forward, full) before a new schema is accepted.

Compatibility mode	Meaning
Backward	New schema can read data written with the old schema. Safe for consumer upgrades first.
Forward	Old schema can read data written with the new schema. Safe for producer upgrades first.
Full	Both directions. Most restrictive. Safest.

Practical approach: Use Protobuf or Avro (not raw JSON) for Kafka messages. Register schemas. Enforce backward compatibility. Deploy consumers before producers when adding fields.

H. API Design

H1. REST vs gRPC vs GraphQL

Dimension	REST	gRPC	GraphQL
Protocol	HTTP/1.1 or 2	HTTP/2 (binary)	HTTP
Format	JSON (text)	Protobuf (binary)	JSON
Contract	Implicit (OpenAPI optional)	Strict .proto schema	Typed schema
Performance	Good	Best	Good
Browser support	Yes	Limited (needs proxy)	Yes
Streaming	SSE / WebSocket	Native bidirectional	Subscriptions
Best for	Public APIs, web, mobile	Internal microservices	Complex client queries
Over-fetching	Common problem	Structured response	Solved

H2. Pagination Patterns

Pattern	How	Use when
Offset	?page=3&size=20	Total count needed; stable sort
Cursor	?after=cursor_id	Infinite scroll; items can be inserted
Keyset	?after_created_at=...&after_id=...	Large tables; efficient via compound index

Why offset pagination breaks at scale: `OFFSET 10000 LIMIT 20` makes Postgres scan and discard 10,000 rows. Cursor pagination uses an index to start exactly where you left off — $O(\log n)$, not $O(n)$.

H3. API Gateway

Handles cross-cutting concerns at the edge so individual services don't have to:

- Authentication & authorisation
- Rate limiting

- Request routing & load balancing
- Protocol translation (REST → gRPC)
- SSL termination
- Request/response logging

Tools: Kong, AWS API Gateway, NGINX, Envoy.

H4. Service Discovery

The problem: In a dynamic cluster, services spin up and down constantly. IP addresses change. How does Service A know the current address of Service B?

Two models:

Model	How	Examples
Client-side discovery	Client queries a service registry directly, picks an instance, and calls it. Client owns the load balancing logic.	Netflix Eureka, Consul (direct)
Server-side discovery	Client calls a load balancer or proxy. The LB queries the registry and forwards to a healthy instance. Client is unaware of discovery.	AWS ALB + ECS, Kubernetes Services, Envoy

Service registry: A database of service instances and their health status. Services register on startup, deregister on shutdown. Health checks confirm liveness.

DNS-based discovery (Kubernetes): Every Kubernetes Service gets a stable DNS name (`my-service.namespace.svc.cluster.local`). Clients call the DNS name; kube-dns resolves to a healthy pod IP. Simple, no special client library needed.

Interview move: "In Kubernetes I'd use a Service + ClusterIP — DNS-based server-side discovery, zero client changes. Outside Kubernetes, Consul with Envoy sidecars gives the same result with automatic health-check-driven deregistration."

H5. API Versioning

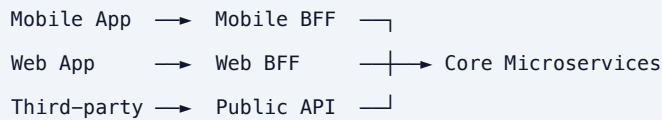
The problem: You need to change an API in a way that would break existing clients. How do you evolve without forcing simultaneous upgrades?

Strategy	Example	Pros	Cons
URL versioning	<code>/api/v1/users</code> <code>/api/v2/users</code>	Explicit, cacheable, easy to document	URL pollution; clients must update imports
Header versioning	Accept: <code>application/vnd.api+json; version=2</code>	Clean URLs	Less visible, harder to test in browser
Query param	<code>/api/users?version=2</code>	Easy to add	Not RESTful; easily forgotten

Deprecation lifecycle: v1 deprecated → 6-month sunset window → traffic monitoring → v1 removed. Never remove without a sunset period and traffic data showing zero usage.

H6. Backend for Frontend (BFF) Pattern

What it is: Instead of one generic API serving all clients (web, mobile, third-party), create a separate backend tailored to each client type.



Why: A mobile app on a 3G connection needs a single coarse-grained API call that returns exactly the fields needed — no over-fetching, no multiple round trips. A web app on broadband can handle finer-grained calls. A generic API is a compromise that serves neither well.

What each BFF does:

- Aggregates multiple microservice calls into one response (eliminates mobile round trips)
- Shapes the response for the specific client (drops unused fields, flattens nested structures)
- Handles client-specific auth flows (mobile uses refresh token rotation; web uses cookies)
- Independently deployable — mobile BFF can change without affecting the web BFF

When not to use it: Small teams. One client type. The API is already simple and well-shaped. BFF adds operational overhead (another service to deploy and monitor per client type).

Interview move: "I'd introduce BFF when mobile and web clients have diverging needs and the generic API is getting cluttered with `?include_fields=` hacks to avoid over-fetching. It's a natural split point when teams own different clients."

I. Resilience Patterns

CIRCUIT BREAKER – STATE MACHINE



Prevents cascading failures: a slow downstream service can exhaust the caller's thread pool. Circuit breaker fails fast instead, preserving capacity. Pair with exponential backoff + jitter on retry.

I1. Circuit Breaker

What it is: Monitor calls to a dependency. When failures exceed a threshold, "open" the circuit — fail fast without calling the dependency. After a reset timeout, "half-open" and test.

State	Behaviour
Closed	Normal; failures counted
Open	Fail immediately; no call made; reset timer running
Half-open	One test request; success → close; failure → open again

Prevents cascading failures where one slow dependency brings down the whole system.

I2. Retry with Exponential Backoff + Jitter

```
delay = base_delay × (2 ^ attempt) + random(0, jitter)
```

Why jitter matters: Without it, all failed requests retry simultaneously, creating a thundering herd on the recovering service. Jitter spreads retries across time.

Max retries: 3–5 is typical. Beyond that, send to a queue rather than blocking the thread.

I3. Bulkhead

What it is: Isolate resources (thread pools, connection pools, semaphores) per dependency. A slow dependency consumes only its own pool, not the shared one.

The ship analogy: watertight compartments mean one flooded section doesn't sink the ship.

I4. Timeout

Rule: Every network call must have a timeout. Never block indefinitely.

- **Connection timeout:** How long to wait to establish a connection (1–5s typical)
- **Read timeout:** How long to wait for a response after connecting (domain-specific)

Interview move: "I'd set the timeout at the 99th percentile of expected response time, not the average. Waiting for a slow p99 is acceptable; waiting for a hung connection indefinitely is not."

I5. Health Checks — Liveness vs Readiness

These are two distinct probes that serve different purposes. Confusing them causes production incidents.

Probe	Question it answers	On failure
Liveness	Is this process alive and not deadlocked?	Kubernetes restarts the container
Readiness	Is this instance ready to serve traffic right now?	Kubernetes removes it from the load balancer pool — does NOT restart

Why the distinction matters:

- During startup, a server may be alive (process running) but not ready (DB migration still running, cache warming). Readiness keeps it out of rotation until it's ready. Without readiness, the LB routes traffic to a server that returns 500s for 30 seconds on every deploy.
- During a graceful shutdown, mark readiness as failing first. The LB drains existing connections. Then the process exits. Without this, in-flight requests are dropped on every deploy.

What each endpoint should check:

Probe	Check
/healthz (liveness)	Can the process respond at all? Just return 200. No dependency checks.
/readyz (readiness)	Is the DB connection pool healthy? Are required caches warm? Are background jobs running?

16. Deployment Strategies**Blue-Green Deployment:**

Two identical production environments — Blue (live) and Green (new version). Deploy to Green while Blue serves all traffic. Switch the load balancer to Green. If Green has issues, switch back to Blue in seconds.

- Zero-downtime deploy
- Instant rollback
- Cost: running two full environments simultaneously

Canary Deployment:

Route a small percentage of traffic (1–5%) to the new version. Monitor error rate, latency, and business metrics. Gradually increase the percentage if metrics are healthy. Roll back by sending 0% to the new version.

- Lower risk than full deploy — only a fraction of users see a bad version
- Requires traffic splitting at the load balancer or service mesh level
- Requires good monitoring — you need to detect problems in the canary slice

Feature Flags:

Deploy code that is off by default. Enable it for specific users, cohorts, or percentages via a config service (LaunchDarkly, GrowthBook, Unleash) without deploying new code.

- Decouples deploy from release — merge anytime, ship when ready
- Enables A/B testing and gradual rollouts
- Operational cost: flag debt accumulates; old flags must be cleaned up

Strategy	Rollback speed	Traffic exposure	Cost
Standard deploy	Minutes (redploy)	100% on bad code	Low
Blue-green	Seconds (LB switch)	100% (but instant rollback)	2× infra cost
Canary	Immediate (0% canary)	1–5% on bad code	Small overhead
Feature flag	Immediate (toggle off)	Only enabled users	Flag management overhead

17. Disaster Recovery — RTO and RPO

Two numbers every architect must know. Interviewers ask these directly.

RTO (Recovery Time Objective): How long can the system be down before the business is materially harmed? The maximum acceptable downtime from failure to recovery.

RPO (Recovery Point Objective): How much data loss is acceptable? The maximum time gap between the last good backup and the failure event.

Scenario	RTO	RPO
Banking core system	Minutes	Zero (no data loss tolerated)
E-commerce checkout	30 min	5 min
Internal analytics dashboard	4 hours	24 hours
Dev/staging environment	Days	Days

DR Strategies (from cheapest to most robust):

Strategy	RTO	RPO	Cost	How
Backup & Restore	Hours–days	Hours	Lowest	Periodic backups to S3; restore on failure
Pilot Light	10–30 min	Minutes	Low	Minimal standby (DB replica running, app servers off); start app servers on failure
Warm Standby	Minutes	Seconds	Medium	Scaled-down replica of production always running; scale up on failure
Active-Active (Multi-region)	Near zero	Near zero	Highest	Full production in both regions; traffic splits; instant failover

Interview move: "I'd start by asking: what are the RTO and RPO requirements? Those two numbers drive the entire DR architecture. A 4-hour RTO can use backup-restore. A 5-minute RTO requires warm standby at minimum. Zero RPO requires synchronous replication, which trades write latency."

J. Security & Auth

J1. JWT + Refresh Token Pattern

JWT structure: `header.payload.signature` (base64, dot-separated). Server validates signature without a DB lookup — stateless.

Token	TTL	Use
Access token	15 min	API auth on every request
Refresh token	7–30 days	Stored server-side; used only to get new access tokens

Why short access TTL: If stolen, it expires in 15 minutes. Without a revocation blocklist, you can't invalidate it — shorter TTL = smaller attack window.

J2. OAuth 2.0 / OIDC

Term	Purpose
OAuth 2.0	Delegated authorisation framework. "Allow this app to read my Google Drive."
OIDC	Authentication layer on OAuth 2.0. Returns an ID token (who you are).
Authorization code flow	Standard for web apps. Code exchanged server-side for tokens.
PKCE	Extension for public clients (SPAs, mobile). Prevents code interception.

J3. mTLS (Mutual TLS)

Both client and server present certificates during the TLS handshake. Both authenticate each other.

Use for: service-to-service communication inside a cluster. Service mesh (Istio, Linkerd) handles mTLS automatically without application code changes.

J4. Secrets Management

The problem: Your application needs database passwords, API keys, and certificates. Hardcoding them in source code or `.env` files in production is a security incident waiting to happen.

Approach	Risk	Use
Hardcoded in code	Leaked in git history forever	Never
<code>.env</code> file on server	Leaked if server is compromised; no rotation	Dev only
Environment variables from CI/CD	Better; no file on disk	Acceptable for low-sensitivity
Secrets manager	Centralised, audited, rotatable, least-privilege access	Production

Tools:

- **AWS Secrets Manager** — managed, auto-rotation, IAM-scoped access, \$0.40/secret/month
- **HashiCorp Vault** — self-hosted, powerful, complex to operate; dynamic secrets (generates DB

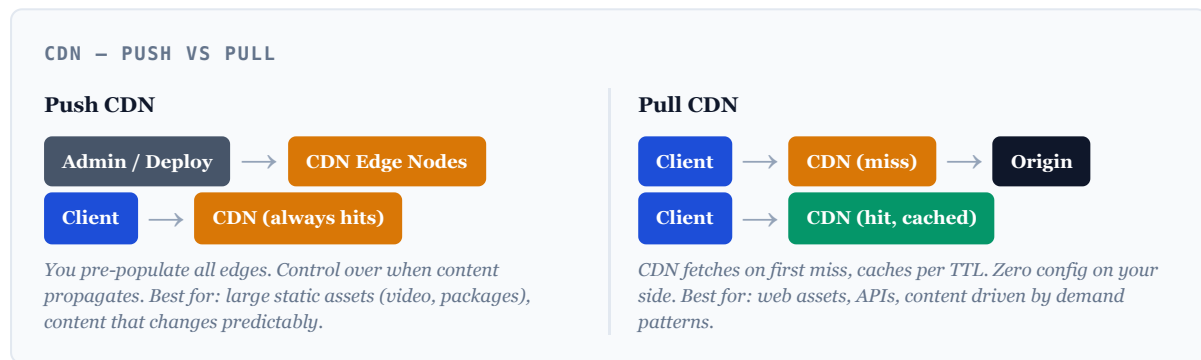
creds on demand with TTL)

- **Kubernetes Secrets** — base64-encoded (not encrypted by default); encrypted at rest with KMS; works well within a cluster

Dynamic secrets (Vault): Instead of a static password, Vault generates a unique DB credential for each app instance with a TTL. On TTL expiry, the credential is revoked. Compromise of one credential is contained to one instance and one TTL window.

Interview move: "I'd never put secrets in environment variables for production. I'd use AWS Secrets Manager with IAM role-based access — the instance role grants permission to read specific secrets, no credentials needed to fetch credentials."

L. Geographic Distribution



L1. Active-Active vs Active-Passive Multi-Region

Model	How	Write consistency	Use when
Active-Passive	One region is primary (handles writes). Other region(s) are replicas (read-only). On primary failure, promote a replica.	Strong (all writes to one region)	Simpler; acceptable failover RTO of minutes
Active-Active	All regions accept reads and writes simultaneously. Changes replicated asynchronously between regions.	Eventual (cross-region replication lag)	Highest availability; global write throughput

The active-active conflict problem: User A in Mumbai and User B in London both update the same record simultaneously in their local regions. Replication delivers both updates to each region. Which one wins?

Conflict resolution strategies:

- **Last-write-wins (LWW)** — the write with the latest timestamp wins. Simple. Risk: clock skew means the "latest" timestamp is not always the logically last write.
- **Application-level resolution** — the app receives both versions and resolves (e.g., shopping cart: merge both carts; bank balance: flag for human review).
- **CRDTs (Conflict-free Replicated Data Types)** — data structures designed to merge without conflict (counters, sets, maps). No conflict to resolve.

L2. Geo-Routing

Routing a user's request to the nearest region or the region best suited to serve them.

Method	How	Use when
Latency-based routing	DNS resolves to the endpoint with lowest measured latency from the user's location	Global user base; want lowest RTT
Geolocation routing	DNS resolves based on geographic region of the request IP	Data residency requirements (EU data stays in EU)
Failover routing	Primary region serves all traffic; secondary becomes active only on health check failure	Active-passive setup

Data residency: GDPR requires EU user data to remain in EU. Geolocation routing + region-isolated data stores enforces this. Users in the EU always hit EU endpoints; their data never crosses to US regions.

M. Storage Patterns

M1. Object Storage (S3)

What it is: Flat, key-value storage for binary objects (files). Infinitely scalable, cheap, durable (11 nines). Not a database — no queries, no transactions, no updates (only put/get/delete by key).

Use when: Images, video, audio, documents, backups, data lake. Anything that's read as a whole file. Write-once, read-many.

Key patterns:

Presigned URLs: Instead of routing file downloads through your server, generate a time-limited URL that lets the client download directly from S3. Your server CPU and bandwidth are not involved.

```
# Server generates:
url = s3.presigned_url('get_object', Bucket='...', Key='...', ExpiresIn=3600)
# Client downloads directly from S3 — server is not in the loop
```

Multipart upload: For files > 100MB, split into parts and upload concurrently. Benefits: parallel upload (faster), resumable (restart failed parts without re-uploading completed parts), no single-request size limit.

Lifecycle policies: Automatically transition objects to cheaper storage tiers (S3 Standard → S3 Infrequent Access → S3 Glacier) based on age. Delete old objects automatically. Critical for cost control on growing data lakes.

S3 vs Database — when each wins:

Situation	Use S3	Use DB
Store a video file	✓	✗
Store video metadata (title, duration, upload_date)	✗	✓
Query "all videos uploaded in the last 7 days"	✗	✓
Serve a file to millions of users	✓ (+ CDN)	✗

Interview move: "I'd store the file in S3 and the metadata (filename, size, content-type, S3 key) in Postgres. Serve the file via a presigned URL or CDN — never stream it through the application server."

M2. Database Indexing

Missing this is the most common gap for engineers who know SQL but not its internals.

B-Tree index (default): Tree structure sorted by the indexed column. Supports: equality (=), range (> , < , BETWEEN), ordering (ORDER BY). Used for almost everything.

Hash index: Maps the exact value to a row pointer. O(1) lookup. Supports only equality (=). Cannot do range queries or sorting. Rarely the right choice — B-tree handles equality too, and also handles ranges.

Composite index: An index on multiple columns. `CREATE INDEX ON orders(user_id, created_at)`. The leftmost prefix rule: a composite index on (A, B, C) can be used for queries filtering on A, A+B, or A+B+C — but not B alone or C alone.

Partial index: An index that only includes rows matching a condition. `CREATE INDEX ON orders(created_at) WHERE status = 'PENDING'`. Smaller, faster, cheaper — covers only the rows you actually query.

Covering index: An index that includes all columns a query needs. The query is answered entirely from the index without touching the table rows at all (index-only scan). Example: `CREATE INDEX ON orders(user_id) INCLUDE (total, status)`.

When an index makes things worse:

- Tables with < 1,000 rows — sequential scan is faster than index overhead
- Very high write rate — every write must update all indexes on the table
- Low-cardinality columns (boolean, status with 3 values) — the DB reads half the table anyway, skip the index

Interview move: "Before adding an index I'd run `EXPLAIN ANALYZE` to see the actual query plan and where the time goes. An index on a foreign key is almost always right. An index on a low-cardinality column is almost always wrong."

M3. SQL Tuning — Schema and Query Level

Indexing is not the only lever. When `EXPLAIN ANALYZE` shows a slow query but indexes are already in place, these are the next checks:

Schema choices that compound at scale:

Choice	Guidance
<code>CHAR(n)</code> vs <code>VARCHAR(n)</code>	Use <code>VARCHAR</code> — <code>CHAR</code> pads to fixed length; wastes disk on short strings
<code>VARCHAR(255)</code>	Size the column to reality; oversized columns waste memory in sort/group operations everywhere
<code>TEXT</code> for short strings	<code>TEXT</code> and <code>VARCHAR</code> are equivalent in Postgres; fine — but use <code>VARCHAR(n)</code> on MySQL where <code>TEXT</code> disables some optimisations
<code>DECIMAL</code> for money	Never <code>FLOAT</code> for currency — floating-point precision errors compound. Use <code>DECIMAL(19,4)</code> or store as integer cents.
Storing binary blobs in DB	Move to S3. BLOBs bloat the DB, skip the cache, and can't be indexed.

Query-level tuning:

- **Avoid `SELECT *`** — fetch only the columns you need; wide rows fill buffer pages faster
- **Avoid joins you can avoid** — for read-heavy paths, denormalise or use a covering index
- **Partition hot tables** — if a table has a "hot" time range (last 7 days of orders), partition by date and route queries to the hot partition only
- **Use the query cache carefully** — Postgres doesn't have a built-in query cache (unlike MySQL); rely on Redis/application caching instead
- **Rewrite correlated subqueries as JOINS** — `SELECT ... WHERE id IN (SELECT id FROM ...)` rewritten as a JOIN can use index-nested-loop instead of scanning per row

The tuning order of operations:

1. `EXPLAIN ANALYZE` — see the actual plan and row estimates
2. Add a missing index
3. Rewrite the query (JOIN instead of subquery, pagination)
4. Denormalise or materialise
5. Partition the table
6. Archive old data

K. Observability

K1. The Three Pillars

Pillar	What	Tools
Logs	Timestamped events. "Order 123 failed at 14:32."	ELK Stack, CloudWatch, Datadog
Metrics	Numeric measurements over time. QPS, error rate, p99 latency.	Prometheus + Grafana, Datadog
Traces	End-to-end request path across services. Where did the time go?	Jaeger, Zipkin, OpenTelemetry

Correlation ID: A unique ID assigned at the edge, passed through every service call and log line. Lets you reconstruct the full journey of any single request.

K2. RED Method (for services)

Letter	Metric	Target
R ate	Requests per second	Baseline × peak multiplier
E rror rate	% requests returning 5xx	< 0.1%
D uration	Latency distribution (p50, p95, p99)	SLO-driven

K3. SLI / SLO / SLA

Term	Definition	Example
SLI	The metric you measure	p99 latency
SLO	Internal target	p99 < 500ms, 99.9% of the time
SLA	External contract with penalty	99.9% uptime; refund if breached

Error budget: SLO 99.9% → 0.1% downtime budget = 8.7 hrs/year. When the budget is consumed, new deploys pause until it recovers.

N. Availability & Reliability

N1. Availability — The Nines

Availability	Downtime / year	Downtime / month
99% (two nines)	87.6 hours	7.3 hours
99.9% (three nines)	8.76 hours	43.8 minutes
99.99% (four nines)	52.6 minutes	4.4 minutes
99.999% (five nines)	5.26 minutes	26 seconds

Availability in series: System is available only when ALL components are up. If component A = 99.9% and component B = 99.9%, combined = $99.9\% \times 99.9\% = 99.8\%$. Each dependency reduces overall availability.

Availability in parallel (redundancy): System is available when ANY component is up. If both nodes fail = $0.1\% \times 0.1\% = 0.01\%$ chance both fail simultaneously. Combined availability = $1 - (0.001 \times 0.001) = 99.9999\%$. Redundancy dramatically increases availability.

Interview move: "Adding a dependency always reduces availability — the combined SLA is the product. If I'm promised 99.99% for my service, I must budget my dependencies' failure rates. Three

99.9% dependencies in series = ~99.7% — I've burned my budget before writing a line of code."

N2. Availability vs Reliability vs Fault Tolerance

Term	Meaning	Example
Availability	% of time the system responds to requests	99.9% uptime
Reliability	System performs its function correctly over time	Answers are always accurate, not just returned
Fault Tolerance	System continues operating despite component failures	One node dies; traffic shifts to healthy nodes with no user impact

A system can be highly available but unreliable (returns 200 but with wrong data). A system can be reliable but not fault-tolerant (single node, always correct, but one crash = full downtime).

O. Architecture Patterns

O1. Monolith vs Microservices

Start here before choosing microservices:

Dimension	Monolith	Microservices
Deployment unit	One deployable artifact	One per service
Team ownership	Shared codebase	Each team owns their service
Scaling	Scale the whole app	Scale individual services independently
Development speed (early)	Fast — no network calls, simple local dev	Slow — service boundaries, networking, contracts
Development speed (at scale)	Slow — merge conflicts, coordination overhead	Fast — teams deploy independently
Operational complexity	Low	High — service discovery, distributed tracing, eventual consistency
Data management	Single DB; easy transactions	Each service owns its DB; distributed transactions are hard
Failure isolation	One bug can take down everything	Failure in one service doesn't affect others (if circuit breakers in place)
When to choose	Early stage, small team, < 5 engineers	Large org, multiple teams, need independent deployability

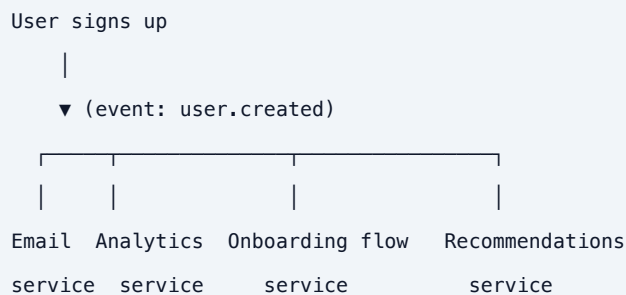
The Modular Monolith: The middle ground. A single deployable artifact internally structured into well-defined modules with enforced boundaries. Modules can't import each other's internals — only public APIs. Easier to split into microservices later when the boundaries are already clean.

The Distributed Monolith (anti-pattern): Microservices that are so tightly coupled they must deploy together. You get all the operational complexity of microservices with none of the independence benefits. Happens when services share a database, have synchronous chains across 5 services, or have no clear domain ownership.

Interview move: "I'd always start with a monolith and extract microservices only when there's a specific pain point — a team stepping on another team's deploys, a specific service that needs 10× the compute of everything else, or a clear domain boundary that's causing merge conflicts. Never extract based on speculation."

O2. Event-Driven Architecture (EDA)

What it is: Components communicate by producing and consuming events rather than calling each other directly. Producers emit events without knowing who consumes them. Consumers react to events independently.



Key properties:

- **Loose coupling** — producer doesn't know consumers exist; consumers don't know about each other
- **Async** — producer doesn't wait for consumers; returns immediately
- **Independently scalable** — each consumer scales based on its own load
- **Resilient** — if one consumer is down, events queue up and are processed when it recovers

EDA vs Request-Response:

	Request-Response (REST/gRPC)	Event-Driven
Coupling	Tight — caller must know callee's address	Loose — producer knows only the event broker
Latency	Synchronous — caller waits	Async — fire and forget
Error handling	Caller handles failure	Events retry from queue; DLQ for poison messages
Use when	Need immediate response	Downstream work can happen later; multiple consumers needed

When EDA is the right call: User registration (send email + update analytics + start onboarding — all independently). Order placed (fulfillment + inventory + notification — parallel and decoupled).

Audit logging — every action emits an event; the audit consumer writes them. CDC — DB change events fan out to multiple subscribers.

O3. MapReduce

What it is: A programming model for processing large datasets in parallel across a cluster. Two phases: **Map** (transform each record independently) and **Reduce** (aggregate the mapped results).

Input: 1 TB of web server logs

Map phase (runs on every machine in parallel):

For each log line → emit (url, 1)

Reduce phase (aggregates by key):

For each url → sum all the 1s → (url, total_hits)

Output: URL hit counts across the entire 1 TB dataset

Why it scales: The Map step is embarrassingly parallel — each record processed independently, no coordination. The framework (Hadoop, Spark) handles data splitting, scheduling, and failure recovery automatically.

Map → Shuffle → Reduce:

1. **Map** — Workers transform input records into (key, value) pairs
2. **Shuffle** — Framework groups all values by key, routes each group to the same Reducer
3. **Reduce** — Aggregates all values for each key

When to reach for it:

- Batch processing over massive datasets (TB–PB scale)
- You can express the problem as: process each record → group by key → aggregate per group
- One-time or scheduled jobs (not real-time)

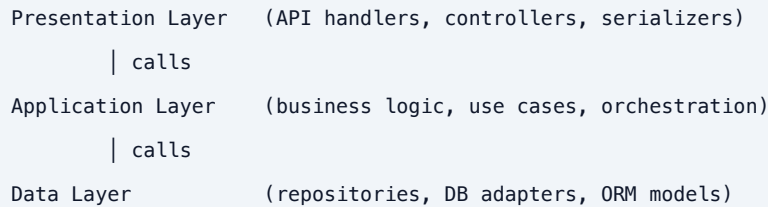
Modern equivalent: Apache Spark replaced MapReduce in most use cases. Spark keeps data in memory between stages instead of writing to disk between Map and Reduce — 10–100× faster for iterative algorithms. The MapReduce mental model still applies.

When it's the wrong tool: Real-time processing (use Kafka Streams, Flink, or Spark Streaming). Simple queries over structured data (use a columnar DB like ClickHouse). Interactive analytics (too slow for on-demand queries).

Interview move: "MapReduce fits batch aggregation over enormous datasets. For real-time aggregation — like a trending topics counter — I'd use stream processing (Kafka + Flink) instead. The key question is always: does the job need to finish in minutes (batch) or milliseconds (stream)?"

O4. Layered / N-Tier Architecture

What it is: Application structured in horizontal layers where each layer communicates only with the layer directly below it.



Rules: Upper layers depend on lower layers, never the reverse. Each layer is independently replaceable.

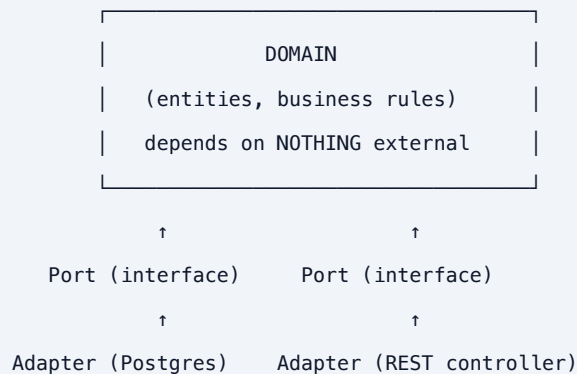
When it works: Standard CRUD services, small teams, code that needs to be immediately navigable by a new engineer.

Where it breaks down: Business logic leaks into the presentation layer (fat controllers). DB row shapes bleed all the way up to the API response. As the system grows, layers become dumping grounds rather than principled boundaries.

Interview move: "I'd use layered architecture for a straightforward CRUD service. Once business rules start varying significantly — pricing tiers, complex validation, permission policies — I'd shift toward hexagonal so the domain code doesn't depend on the DB."

O5. Hexagonal Architecture (Ports & Adapters) / Clean Architecture

What it is: Invert the dependencies so the domain (business rules) depends on nothing external. Infrastructure (DB, HTTP, queues) plugs in via adapters defined by the domain.



Ports — interfaces the domain defines ("I need a SubmissionRepository that can `save` and `find_by_id`"). **Adapters** — concrete implementations (PostgresSubmissionRepository, InMemorySubmissionRepository for tests).

Why it matters:

- Swap Postgres for DynamoDB → write one new adapter, domain untouched.
- Test business logic with zero infrastructure — stub the port, no DB required.
- The most important code (domain rules) is testable in complete isolation.

Your example (Munshi): Vendor reconciliation rules live in the domain. The GSTR-2A CSV parser and the SQLite adapter are both adapters. The domain only knows the `VendorRepository` interface — not which database backs it.

Interview move: "Hexagonal is my default for anything with complex, frequently-changing business rules. The payoff is testability of the domain without a database. The cost is more files and more interfaces — worth it once the domain has more than a handful of core rules."

O6. Domain-Driven Design (DDD)

What it is: A design approach where the model shared between engineers and domain experts drives all architecture decisions. The code vocabulary matches the business vocabulary exactly.

Core concepts:

Concept	What it means
Bounded Context	An explicit boundary where a domain model is defined and consistent. "Account" means different things in Billing vs. Auth — those are separate bounded contexts with their own models.
Aggregate	A cluster of domain objects always consistent together. One root entity owns the aggregate; all changes go through the root. <code>Order</code> is the root; <code>OrderItem</code> lives inside and is never accessed directly.
Entity	An object with identity that persists over time (User, Order). Equality is by ID, not by value.
Value Object	An object with no identity; equality is by value (Money, Address, Email). Immutable.
Domain Event	A record of something that happened. <code>OrderPlaced</code> , <code>ExamClosed</code> . Always past tense. Drives integration between bounded contexts.
Repository	Interface through which aggregates are loaded and saved. Domain doesn't know if backing store is Postgres, Redis, or a file.
Anti-Corruption Layer (ACL)	Translation layer between two bounded contexts. Prevents one model's concepts from polluting another's.
Ubiquitous Language	Shared vocabulary used in all conversations and in the code itself. Engineers and domain experts talk the same way.

Context Map — how bounded contexts integrate:

Integration style	Description
Published Language	Open standard format (JSON API, Protobuf). Any context can consume.
Customer/Supplier	Supplier team produces; customer team consumes and drives requirements.
Conformist	Downstream conforms to upstream model entirely; no translation.
ACL	Downstream translates upstream model via its own translation layer.
Shared Kernel	Two teams share a small common model. High coupling; requires coordination.

When DDD pays off: Complex domains with many rules that change frequently. Multiple teams owning different parts of the system. Business vocabulary is large and precise (finance, healthcare, legal, compliance).

When it's overkill: Simple CRUD. Internal tooling. Small teams where a domain expert isn't in the room.

Your example (Trade Compliance): HS codes, tariff classifications, duty rates, and trade restrictions are a complex domain with precise vocabulary. A `TariffRule` aggregate with `apply_to(shipment)` is a natural DDD model. The Researcher agent and Writer agent map to different bounded contexts within the compliance domain.

Interview move: "DDD pays off when the domain is genuinely complex — meaning the business rules are the hard part, not the infrastructure. For a product catalogue: layered architecture is fine. For a billing system with pricing rules, trials, proration, refunds, and tax jurisdiction logic — I'd want bounded contexts and aggregates so the code reflects how the business actually thinks."

07. Strangler Fig Pattern

What it is: Incrementally replace a monolith by routing new traffic to new services while the old monolith handles the rest. The new system grows around the monolith until the monolith can be switched off — like a strangler fig vine that eventually replaces its host tree.

Phase 1 – Monolith handles everything:

Client → Monolith

Phase 2 – Proxy introduced; one feature migrated:

Client → Proxy (nginx / API Gateway)

```

└─ /auth/*      → New Auth Service
└─ /*          → Monolith (everything else)
  
```

Phase 3 – More features migrated:

Client → Proxy

```

└─ /auth/*      → Auth Service
└─ /payments/*  → Payments Service
└─ /notify/*    → Notification Service
└─ /*          → Monolith (shrinking core)
  
```

Phase 4 – Monolith retired:

Client → API Gateway → New services only

Key enabler — the proxy: Without a proxy/API gateway, you'd need a hard cutover. The proxy lets you shift traffic incrementally, feature by feature, without touching the monolith's code.

How to choose what to migrate first:

1. Features with the clearest bounded context (auth, billing, notifications)
2. Features that need independent scaling (the CPU-intensive code execution in CodeMas)
3. Features causing the most merge conflicts between teams

Data migration (the hard part): The monolith owns the database. Options:

- **Shared DB (short-term):** New service reads/writes the monolith's DB. Breaks independence but

gets you running fast.

- **DB per service (target):** New service owns its data, syncs via events or CDC. The eventual goal.

Interview move: "I'd never greenfield-rewrite a working monolith. Strangler Fig is the only safe approach — incremental, each step is rollback-safe, and you can stop at any point. The proxy layer is the critical first investment. Once it's in place, you can move features one at a time without a big bang."

O8. Serverless Architecture

What it is: Code deployed as stateless functions triggered by events. No server management — the cloud provider handles scaling, patching, and availability. You pay per invocation, not per idle hour.

Mental model: Function = handler that runs when triggered, completes, and exits. State lives in external storage (DB, cache, queue).

Property	Serverless	Traditional server
Startup	Cold start: 50ms–5s	Always warm
Scaling	Automatic (0 → thousands)	Manual or autoscale with warm floor
State	Stateless — stored externally	Can hold in-process state
Cost model	Per invocation + duration	Per hour (idle costs money)
Long-running tasks	Limited (15 min max on Lambda)	Unlimited
Debugging	Harder — distributed, no SSH	Easier — local, consistent

Cold start problem: First invocation after inactivity: download package, initialise runtime, run init code. 2–5 seconds for large runtimes (Java, .NET). Solutions: provisioned concurrency (keep N warm), smaller packages, lightweight runtimes (Node, Python).

When serverless wins:

- Event-driven, bursty workloads (code execution, image resizing, webhook handling)
- Low-traffic services that would waste money on an always-on server
- Glue code between AWS services (S3 trigger → process → DynamoDB)

When serverless loses:

- Long-running tasks (DB migrations, video encoding)
- Needs in-process state or background threads
- Very high steady-state traffic (per-invocation cost exceeds EC2)
- Cold-start latency is user-facing

Your proof point (CodeMas): Lambda is the code sandbox. Students submit → SQS buffers → Lambda runs student code in isolation per invocation. Perfect fit: bursty, stateless, hard time limit (15s), VPC-isolated per invocation.

Interview move: "I'd use serverless for the event-driven edge of a system — processing queue jobs, responding to S3 uploads, webhooks. For anything with sustained high-throughput traffic or long-running work, I'd rather pay for an always-warm container or EC2 instance. The inflection point is roughly when the per-invocation cost exceeds what a reserved instance would cost."

P. Infrastructure Fundamentals

P1. DNS — How Domain Names Resolve

Why architects need this: Every system design starts with users typing a URL. DNS is step one.



Key record types:

Record	Purpose	Example
A	Domain → IPv4 address	api.example.com → 93.184.216.34
AAAA	Domain → IPv6 address	api.example.com → 2600:1404::1
CNAME	Domain → another domain (alias)	www.example.com → example.com
MX	Mail server for this domain	example.com → mail.example.com
TXT	Arbitrary text (SPF, DKIM, verification)	"v=spf1 include:sendgrid.net ~all"

TTL (Time To Live): How long DNS resolvers cache the answer. Low TTL (60s) = fast failover but more DNS traffic. High TTL (86400s) = less DNS traffic but slow propagation of IP changes.

DNS for load balancing: Round-robin DNS — return multiple A records; clients pick one. Not true load balancing (no health checks, sticky client caches), but useful for very basic distribution.

P2. Virtual Machines vs Containers

	Virtual Machine	Container
Isolation level	Full OS — own kernel, own drivers	Process-level — shares host kernel
Startup time	Minutes (boot OS)	Seconds (start process)
Size	GBs (full OS image)	MBs (app + deps only)
Overhead	High — hypervisor + full OS per VM	Low — thin layer above host kernel
Security boundary	Strong — separate kernel	Weaker — kernel is shared
Use when	Hard multi-tenancy, different OS needed, strong isolation required	Microservices, CI/CD, Kubernetes, fast scaling

Hypervisor: Software that creates and runs VMs. Type 1 (bare-metal): runs directly on hardware (VMware ESXi, KVM). Type 2 (hosted): runs on a host OS (VirtualBox, VMware Workstation).

Why containers won for microservices: Seconds to start vs minutes. Dozen MB image vs GB disk. Docker + Kubernetes makes orchestration, scaling, and rolling deploys practical at scale. VMs still win where you need hard security boundaries (cloud multi-tenancy) or a different OS.

P3. TCP vs UDP

Two transport-layer protocols. You must know when each is appropriate — it comes up in every real-time system design question.

	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Delivery guarantee	Guaranteed — retransmits lost packets	Best-effort — packets may be dropped
Ordering	Guaranteed in-order delivery	No ordering guarantee
Flow control	Yes — slows sender to match receiver	No
Overhead	Higher (headers + state + ACKs)	Lower (minimal header, no state)
Latency	Higher (retransmits add latency)	Lower

Use TCP when: All data must arrive and in order. HTTP, database connections, file transfer, email, SSH.

Use UDP when: Speed matters more than guaranteed delivery. Late data is worse than no data. Real-time audio/video (VoIP, video calls), live game state, DNS queries, metric telemetry.

The video call example: A dropped video frame is visible for 33ms then gone. Waiting for a retransmit takes 50–300ms and causes a freeze — worse than the dropped frame. UDP + application-level error concealment is the right choice.

HTTP/3 (QUIC): HTTP/3 runs over QUIC, which is built on UDP. It implements its own reliable delivery, connection migration, and congestion control — getting TCP-like reliability without TCP's

head-of-line blocking. Used by YouTube, Google, Cloudflare.

P4. Unique ID Generation at Scale

Generating globally unique IDs across a distributed system is a classic interview problem.

Requirements: unique, (optionally) sortable by time, no coordination bottleneck.

Option 1 — UUID (Universally Unique Identifier):

- 128-bit random value. Effectively zero collision probability.
- No coordination — each node generates independently.
- Problem: not sortable. Random writes into B-tree indexes cause page splits (random I/O). Bad for high-write tables.
- Use when: you need uniqueness without any ordering requirement.

Option 2 — Database auto-increment:

- Simple. Sequential. Sortable.
- Single point of coordination — the database is the bottleneck.
- Fails when you shard (each shard generates its own sequence; no global ordering).

Option 3 — Snowflake ID (Twitter, 2010):

A 64-bit integer composed of:

```
| 41 bits timestamp (ms since epoch) | 10 bits machine ID | 12 bits sequence |
```

- Sortable by time — higher ID = newer
- No coordination — machine ID is assigned once at startup
- Generates up to 4,096 unique IDs per millisecond per machine
- Epoch of ~69 years before overflow (with a custom epoch start)

Option 4 — ULID (Universally Unique Lexicographically Sortable Identifier):

- 128-bit: 48-bit timestamp + 80-bit random
- Lexicographically sortable (unlike UUID v4)
- URL-safe encoding
- No machine ID configuration needed

When it comes up in interviews: Any system requiring user-generated content, orders, messages, events. The question tests whether you know why random UUIDs hurt write performance at scale.

Interview move: "I'd use Snowflake IDs for primary keys on write-heavy tables — sortable, no DB coordination, and the timestamp embedded in the ID gives me partition routing by time for free."

Q. Location-Based System Patterns

Q1. Geohashing

What it is: Encodes a geographic coordinate (latitude, longitude) into a short alphanumeric string. Nearby locations share a common prefix. Prefix length = precision.

```
San Francisco (37.7749, -122.4194) → "9q8yy"
A nearby block                      → "9q8yx"  ← shares prefix "9q8y"
New York City                      → "dr5ru"  ← different prefix entirely
```

Why the prefix property matters: To find all restaurants within 1km of the user, query all geohash cells with the same 5-character prefix — one range scan in the DB instead of a `WHERE ST_Distance(...) < 1000` full table scan.

Precision table:

Prefix length	Cell size	Use case
4 chars	~40 km × 20 km	Country-level
5 chars	~4.8 km × 4.8 km	City neighbourhood
6 chars	~1.2 km × 0.6 km	Street level
7 chars	~150 m × 150 m	Building level

Edge case — boundary problem: A user at the edge of a geohash cell is close to locations in the neighbouring cell, but they share no prefix. Fix: query the 8 surrounding cells in addition to the user's cell (9 cells total).

How to store and query:

```
-- Store
INSERT INTO places (name, lat, lon, geohash) VALUES ('Cafe', 37.77, -122.41, '9q8yy');
CREATE INDEX ON places(geohash);

-- Query: find nearby places
SELECT * FROM places WHERE geohash LIKE '9q8yy%'; -- fast index range scan
```

Your systems: An Uber-like system stores driver locations with geohash precision 6 (street level). On rider request, compute rider's geohash, query surrounding 9 cells for available drivers.

Q2. Quadrees

What it is: A tree where each node represents a rectangular region. Each node has exactly 4 children — NW, NE, SW, SE. Subdivision continues until each leaf contains at most N points (or reaches minimum cell size).

When quadrees beat geohashing:

- **Dynamic density:** Geohash cells are fixed size. Quadtree cells subdivide where density is high (Manhattan) and stay large where density is low (rural areas). Better for unevenly distributed data.

- **Spatial queries:** "Find all 10 nearest points" is a natural tree traversal. Geohash needs to expand prefix length iteratively until enough results are found.

Use cases: Google Maps tiles (divide the world into quadrants recursively), spatial indexing in game engines, collision detection, 2D range queries.

Interview move: "For most proximity search questions I'd use geohashing — it maps directly to a database range query and is operationally simple. I'd reach for quadrees when density is extremely uneven and fixed-size cells would either be too coarse (rural) or return too many results (urban) at the same precision level."

R. Multi-Tenant Architecture

What it is: A single deployment of software serves multiple distinct customers (tenants). Each tenant's data is isolated from others. The architecture decision is *how* that isolation is enforced — and that choice drives cost, compliance posture, and operational complexity.

R1. The Three Models

Model	Structure	Isolation level	Cost per tenant	Best for
Silo (dedicated)	Each tenant gets their own DB (and often their own compute)	Strongest — hardware-level	High	Healthcare, finance, government, enterprise
Pool (shared)	All tenants share one DB; every table has a <code>tenant_id</code> column	Code-level only	Lowest	SaaS startups, SMB market
Bridge (hybrid)	Shared compute; DB isolation varies by tier (shared DB for free, per-tenant schema for growth, dedicated cluster for enterprise)	Tiered	Tiered	Mid-market SaaS

Silo model:

```
Tenant A → dedicated DB-A
Tenant B → dedicated DB-B
Tenant C → dedicated DB-C
```

- Noisy neighbour is impossible — Tenant A's heavy query cannot slow Tenant B.
- Per-tenant backups, migrations, rollbacks.
- Operational overhead scales linearly with tenant count.
- Cost floor: you pay for idle resources per tenant even if they're inactive.

Pool model:

```
Tenant A ↴
Tenant B └─── shared DB (every table has tenant_id FK)
Tenant C ↵
```

- Cheapest and simplest to start.
- Risk: one missing `WHERE tenant_id = ?` exposes all tenants' data.
- One slow tenant can degrade DB performance for everyone.
- Hard to give enterprise customers a data-residency guarantee.

Bridge model:

```
Free / SMB tier → shared DB, shared compute
Growth tier    → per-tenant schema on shared cluster
Enterprise tier → dedicated DB cluster, dedicated compute
```

This is how Stripe, Salesforce, and Shopify operate at scale.

R2. Row-Level Security (RLS) — The Pool Model's Safety Net

The pool model's biggest risk is a missing `WHERE tenant_id =` clause. Postgres Row-Level Security enforces isolation at the database level — a missing `WHERE` clause returns zero rows instead of all tenants' data.

```
-- Enable RLS on every tenant-scoped table (run once per table):
ALTER TABLE submissions ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON submissions
  USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- Django middleware sets this at the start of every request:
--   SET LOCAL app.tenant_id = '<uuid-from-jwt>'
-- After that, every ORM query is automatically scoped – no WHERE clause needed.
```

Middleware wiring:

1. JWT arrives with `tenant_id` claim.
2. Django middleware extracts `tenant_id` and calls `SET LOCAL app.tenant_id = ?`.
3. All ORM queries on RLS-enabled tables automatically include the tenant filter.
4. **PgBouncer caveat:** Use transaction mode. `SET LOCAL` is transaction-scoped — it resets when the transaction ends, so the next connection's user cannot inherit it. Session mode would leak the tenant context across connections.

R3. Tenant Routing

Subdomain routing (most common):


```
acme.yoursaas.com      →  tenant_id = 'acme'
beta-corp.yoursaas.com →  tenant_id = 'beta-corp'
```

nginx or API Gateway extracts the subdomain, resolves to `tenant_id` , injects into the request context.

Custom domain routing:

```
app.acme.com → CNAME → yoursaas.com → tenant resolved from Host header
```

The `tenants` table stores `custom_domain → tenant_id` . Wildcard TLS cert or per-tenant cert via Let's Encrypt / cert-manager.

Path-based routing:

```
yoursaas.com/t/acme/dashboard
```

Simpler TLS; less professional for enterprise customers.

R4. Tenant Isolation Checklist

Every layer needs isolation, not just the DB:

Layer	Isolation mechanism
Database	RLS + <code>tenant_id</code> FK on every table
Cache (Redis)	Key prefix: <code>{tenant_id}:{key}</code> — never store unscoped keys
File storage (S3)	Path prefix: <code>s3://bucket/{tenant_id}/uploads/</code> + IAM prefix policy
Message queue	SQS FIFO <code>MessageGroupId = tenant_id</code> , or dedicated queue per tenant
Background jobs	Tag every job with <code>tenant_id</code> ; worker sets DB context before executing
Logging	Structured logs always include <code>tenant_id</code> field
Rate limiting	Enforce per <code>tenant_id</code> , not per IP (a shared office = one IP, many tenants)
Search index	Per-tenant index, or <code>tenant_id</code> metadata filter before returning results

R5. The Noisy Neighbour Problem

One tenant's heavy load degrades performance for others on a shared system.

Solutions:

Problem	Fix
One tenant floods the queue	SQS FIFO <code>MessageGroupId = tenant_id</code> — each tenant's messages stay in their own lane
API spam from one tenant	Per-tenant rate limiting: Redis counter <code>rate:{tenant_id}:{window}</code>
One tenant's query hogs DB	Per-session <code>statement_timeout</code> ; the runaway query is killed, not everyone else's
One tenant starves Lambda concurrency	Reserved concurrency per tenant (expensive tier)
One tenant monopolises worker pool	Fair-queue scheduling: background workers pull from per-tenant queues in round-robin

R6. Onboarding and Offboarding

Onboarding:

1. Create record in `tenants` table (name, plan, `tenant_id`).
2. Provision subdomain (or DNS entry for custom domain).
3. Pool model: nothing else DB-side — RLS handles it automatically.
4. Silo model: run schema migration against the new tenant's dedicated DB.
5. Seed required defaults (roles, permissions, plan limits).
6. Send welcome email + first admin invite.

Offboarding / GDPR Article 17 erasure:

1. Deactivate tenant (stop accepting logins, mark `deleted_at`).
2. Export all tenant data (GDPR Article 20: data portability).
3. `DELETE FROM every_table WHERE tenant_id = ?` (or `DROP SCHEMA tenant_{id}` in silo).
4. Delete S3 objects under `{tenant_id}/` prefix.
5. Flush Redis keys matching `{tenant_id}:*` .
6. Revoke subdomain DNS entry.
7. Write erasure event to audit log (tamper-evident).

R7. Interview Move

"Multi-tenancy is a spectrum. I'd start with the pool model — shared DB with `tenant_id` on every table — and enforce isolation at the database with Postgres RLS rather than in application code. That makes a missing WHERE clause return zero rows instead of all tenants' data.

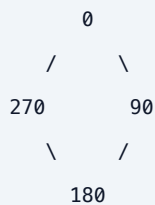
For enterprise customers who need data residency or compliance certifications (SOC 2, HIPAA), I'd provision a dedicated DB cluster (silo). Everyone else stays in the pool. The routing layer doesn't care — it resolves `tenant_id` from the JWT and injects it. That's the bridge model in practice.

The three layers I always check: DB isolation (RLS or dedicated DB), cache key namespacing (`tenant_id` prefix), and queue isolation (`MessageGroupId` or separate queue per tenant to prevent noisy neighbours)."

S. Consistent Hashing

The problem it solves: You have N cache or DB nodes and need to distribute keys evenly. The naive approach — $\text{node} = \text{hash}(\text{key}) \% N$ — breaks badly when N changes: adding or removing one node remaps nearly every key, causing a cache miss storm that hits the database.

Consistent hashing maps both keys and nodes onto a virtual ring (0 to 2^{32}). Each node owns the arc from its position clockwise to the next node. When a node changes, only the $1/N$ keys in that arc are remapped — everything else stays.



Nodes placed at $\text{hash}(\text{node_id})$:

Node A → position 45 (owns arc 315° → 45°)
 Node B → position 135 (owns arc 45° → 135°)
 Node C → position 225 (owns arc 135° → 225°)
 Node D → position 315 (owns arc 225° → 315°)

Key lookup:

$\text{hash}(\text{key1}) \rightarrow 80 \rightarrow \text{walk clockwise} \rightarrow \text{hits Node B} \rightarrow \text{routed there}$
 $\text{hash}(\text{key2}) \rightarrow 200 \rightarrow \text{walk clockwise} \rightarrow \text{hits Node C} \rightarrow \text{routed there}$

When a node is removed:

- Only keys between the previous node and the removed node's position need to remap to the removed node's successor.
- All other keys: unaffected.
- Compare: $\text{hash}(\text{key}) \% N$ remaps $(N-1)/N \approx \text{all keys}$ on any topology change.

Virtual nodes (vnodes): Each physical node is placed at multiple positions on the ring (e.g. 150 per node). This:

- Distributes load more evenly (avoids hot arcs with non-uniform placement).
- Lets higher-capacity nodes own more positions (weighted distribution).
- Ensures smooth key migration when nodes join or leave.

Where it's used:

System	How
Cassandra	Partitions data across nodes using a token ring — consistent hashing
DynamoDB	Internal ring routes requests by partition key
Redis Cluster	16,384 hash slots distributed across nodes; consistent hashing determines which slot → which node
CDN	Route requests to the same edge node for the same URL (cache locality)
Load balancers	Sticky sessions: same user always routes to the same backend

Interview move: "Consistent hashing solves the reshuffling problem when nodes are added or removed from a distributed cache or database. The ring guarantees only $1/N$ keys migrate on a topology change instead of all of them. Virtual nodes handle uneven physical hardware — a node with $2\times$ the RAM gets $2\times$ the ring positions and $2\times$ the keys."

T. Bloom Filters

What it is: A probabilistic data structure for set membership that uses far less memory than a hash set. Answers: "Is X in the set?" with one of two responses:

- **"Definitely not"** — 100% accurate, zero false negatives.
- **"Probably yes"** — small, configurable false positive rate; never false negatives.

How it works:

1. Allocate a bit array of size m (all zeros).
2. Choose k independent hash functions.
3. **Add X:** run X through all k hash functions → set those bit positions to 1.
4. **Check X:** run X through all k hash functions → if ANY bit is 0, X is definitely not in the set. If ALL bits are 1, X is probably in the set (all bits might have been set by previous insertions of other items).

Trade-off: False positive rate is tunable — lower rate → larger bit array. False negatives never occur.

Sizing rule of thumb: 10 bits per expected element → ~1% false positive rate. Formula: $m = -(N \times \ln p) / (\ln 2)^2$ where N = expected items, p = desired false positive rate.

Where it's used:

Use case	Why
Cache miss optimisation	Before querying the DB for a key, check the bloom filter. "Definitely not" → skip the DB entirely. Only query DB when "probably yes". Eliminates DB hits for keys that will never exist.
URL deduplication in web crawlers	"Have we crawled this URL?" False positive = skip a URL (acceptable). False negative = crawl twice (unacceptable). Bloom filter eliminates false negatives. Your example: CodeMas web crawler skeleton in Part 6.
Username availability	Quick bloom filter check before the DB query — most taken usernames are filtered without hitting the DB.
Chrome Safe Browsing	Browser ships with a bloom filter of known-bad URLs. Local check first, no network call unless "probably yes".
Cassandra / LevelDB	SSTable bloom filters: "Does this SSTable definitely not contain this row key?" Skips disk reads for keys that won't be found.

Cannot delete from a standard bloom filter. Deletion requires a counting bloom filter (each bit becomes a counter).

Interview move: "I use a bloom filter as a fast pre-filter in front of an expensive check — DB lookup, network call, disk read. The guarantee is asymmetric: 'definitely not' is always correct and lets me skip the expensive operation; 'probably yes' still needs the real check. In practice, a well-sized bloom filter eliminates 90–99% of unnecessary expensive calls."

U. Feature Flags

What it is: A runtime toggle that enables or disables a code path without a new deployment. Decouples **deploy** (code ships to prod) from **release** (users can see it).

```
if feature_flag("new_checkout_flow", user=request.user):
    return new_checkout(request)
else:
    return old_checkout(request)
```

Types of flags:

Type	Scope	Use case
Kill switch	Global on/off	Instantly disable a feature causing incidents — no hotfix deploy
Gradual rollout	% of users	Roll to 1% → 10% → 50% → 100%, watching error rates at each step
User targeting	Specific users or groups	Beta users, internal employees, specific tenants
A/B test	Random 50/50 split	Measure impact before committing to full rollout
Dark launch	All users, results discarded	Run new code in prod with real data; hide results from users
Operational	Infrastructure switches	Toggle between Postgres and DynamoDB; swap queue implementations

Flag evaluation order (most systems):

1. Is the flag globally disabled? → off for everyone
2. Is this user in the override list? → use override
3. Is this user in a targeted cohort? → enabled (beta users, employees)
4. Is this user in the rollout %? → enabled
5. Default → disabled

Ops pattern — trunk-based development: Feature flags let all engineers commit to `main`. Features are always deployed; flags control who sees them. No long-lived feature branches, no merge conflicts.

Release lifecycle:

```

Build behind flag (default: off)
  → Deploy to prod (zero user impact)
    → Enable for internal team (test with real production data)
      → 1% rollout → watch error rate and latency → 10% → 50% → 100%
        → Remove the flag (most important step — flags are tech debt)
  
```

Flag storage options:

Option	Trade-off
Environment variable	Simple; requires redeploy to change
Database table	No redeploy; cache aggressively (TTL 30s) to avoid DB on every request
Redis	Fast; TTL-based; loses history and targeting rules
Dedicated service (LaunchDarkly, Unleash, Flagsmith)	Real-time evaluation, targeting, analytics — at a cost

Flag debt: Every flag is a branching path. 10 flags = up to 2^{10} code paths to reason about. Treat flag removal as a first-class ticket in the sprint after full rollout.

Interview move: "Feature flags are how I'd do a zero-risk release. The feature ships to prod on day one, hidden behind a flag. Internal team tests it with real production data. Then I roll to 1% of users, watch error rate and latency for 30 minutes, then 10%, then full. If anything spikes, I flip the flag off — no hotfix deploy, no rollback, instant. The cost is flag debt, which I'd manage by creating a cleanup ticket the moment a flag hits 100%."

V. Zero-Downtime Schema Migrations

The problem: You need to change the DB schema while the application is running. During a rolling deploy, old code and new code run simultaneously. Any migration that breaks the currently-running version causes an outage.

The core rule: Every migration must be backward-compatible with the code currently running in production.

The Expand/Contract Pattern

Every breaking schema change is split into three phases, each independently deployable:

Phase 1 – EXPAND

Add the new structure (column, table, index).
Old code ignores it. New code writes to both old and new.
Deploy new code that dual-writes.

Phase 2 – MIGRATE

Backfill existing rows in batches (no table lock).
Verify data is consistent.

Phase 3 – CONTRACT

Old code is gone. Drop the old structure.
New code reads only from the new structure.

Example: Renaming `users.name` → `users.full_name` :

```
-- ❌ WRONG: breaks old code during deploy window
ALTER TABLE users RENAME COLUMN name TO full_name;

-- ✅ RIGHT:

-- Phase 1: Add new column; deploy code that writes to BOTH
ALTER TABLE users ADD COLUMN full_name VARCHAR;
-- New code: INSERT ... (name=val, full_name=val)

-- Phase 2: Backfill; no lock on modern Postgres (UPDATE in batches of 1000)
UPDATE users SET full_name = name WHERE full_name IS NULL;

-- Phase 3: Old code is gone; drop the old column
ALTER TABLE users DROP COLUMN name;
```

Common migrations and how to make them safe:

Operation	Risk	Safe approach
Add column (nullable)	None	Just add it — old code ignores it
Add column (NOT NULL, no default)	Table lock during backfill	Add nullable → backfill in batches → add NOT NULL constraint
Add index	<code>CREATE INDEX</code> takes <code>AccessExclusiveLock</code> — blocks all queries	<code>CREATE INDEX CONCURRENTLY</code> — builds in background, no lock
Rename column	Old code breaks	Expand/Contract: add new name, dual-write, drop old name
Remove column	Old code breaks if it reads the column	Remove from code first (deploy) → then drop from DB (separate migration)
Add foreign key	Scans and locks both tables	Add as <code>NOT VALID</code> → <code>VALIDATE CONSTRAINT</code> in background (Postgres 9.4+)
Backfill large table	Full table scan locks rows	Batch with <code>LIMIT 1000</code> + sleep; use <code>pg_repack</code> for live table rebuilds

Lock levels to know (Postgres):

Operation	Lock	Impact
ADD COLUMN (nullable)	AccessShareLock	Safe — no write blocking
ADD COLUMN NOT NULL (no default)	AccessExclusiveLock	Blocks all queries
CREATE INDEX	AccessExclusiveLock	Blocks all queries
CREATE INDEX CONCURRENTLY	No exclusive lock	Safe; takes longer
ADD CONSTRAINT ... NOT VALID	ShareRowExclusiveLock	Brief; safe for FK
VALIDATE CONSTRAINT	ShareUpdateExclusiveLock	Background; safe

Interview move: "Every schema change I deploy is backward-compatible with the currently-running code version. I use Expand/Contract: add the new structure, dual-write, backfill, then drop the old structure in a separate migration and separate deploy. The only operations I still schedule for low-traffic windows are backfills on tables with hundreds of millions of rows where even batched updates accumulate meaningful lock time."

W. Webhooks

What it is: A reverse API call. Instead of polling an external service for state changes, you register a URL and the service POSTs a JSON payload to you when an event happens.

Traditional polling:

Your app: "Any new payments?"

Razorpay: "No"

Your app: "Any new payments?"

Razorpay: "No"

...8 minutes pass...

Your app: "Any new payments?"

Razorpay: "Yes — here it is"

Webhook:

Razorpay → POST /webhook/payment

Your app → processes → 200 OK

The contract:

1. Register your URL with the provider.
2. Provider POSTs a JSON payload to your URL when the event fires.
3. You return 200 within their timeout (usually 5–30 seconds).
4. Non-200 or timeout → provider retries with exponential backoff.

Design: consuming webhooks correctly

1. Return 200 immediately, process async:

Webhook arrives → write raw payload to internal queue → return 200

Background worker → dequeue → verify signature → deduplicate → process

Never do real work inside the webhook handler. Slow processing → timeout → retry storm → duplicate work.

2. Verify the HMAC signature:

```
import hmac, hashlib

def verify_webhook(request, secret: str) -> bool:
    expected = hmac.new(
        secret.encode(),
        request.body,
        hashlib.sha256
    ).hexdigest()
    received = request.headers.get("X-Signature", "")
    return hmac.compare_digest(expected, received)
```

Always verify before touching anything. Prevents spoofed events.

3. Idempotent processing (providers will retry):

```
event_id = payload["id"]
# Postgres upsert – safe to call multiple times with same event_id
rows = db.execute(
    "INSERT INTO processed_events (id) VALUES (?) ON CONFLICT DO NOTHING",
    event_id
).rowcount
if rows == 0:
    return 200 # already processed, skip silently
# ... process the event
```

4. Fan-out via internal event bus:

One incoming webhook can trigger multiple internal handlers. The webhook receiver only writes to a queue; separate consumers handle email, analytics, Slack notification independently.

Design: sending webhooks (you are the provider)

Use the **Outbox Pattern** to guarantee delivery:

1. In the same DB transaction as the business operation, write the event to an `outbox` table.
2. A background poller reads the outbox and delivers to the customer's endpoint.
3. If the delivery succeeds (200), mark the outbox row as sent.
4. If it fails, retry with exponential backoff. After N failures, move to DLQ, notify customer.

This guarantees the webhook fires if and only if the business operation committed — no fire-and-forget that can silently drop events.

Common failure modes:

Problem	Solution
Provider retries after your timeout	Return 200 fast; process async
Duplicate delivery	Upsert on <code>event_id</code>
Replay attack	Timestamp in payload; reject if > 5 min old
Provider's IP changes	Verify HMAC signature, not source IP
Customer endpoint is down	Exponential backoff + DLQ + customer notification

Interview move: "When consuming: return 200 immediately and queue the event — never process in the webhook handler. Verify the HMAC signature before touching anything. Upsert on `event_id` for idempotency. When sending: use the Outbox Pattern so the webhook fires if and only if the DB transaction committed."

X. Connection Pooling

Why it matters: Opening a Postgres connection costs ~5ms and spawns one backend OS process consuming 5–10 MB RAM. At 1,000 concurrent requests with no pooling: 1,000 backend processes, 5–10 GB RAM just for connections, all competing for CPU. Postgres's default `max_connections` is 100.

What a pool does: Reuse a fixed set of long-lived connections across many short-lived requests. A pool of 25 connections can serve thousands of concurrent requests.

1,000 concurrent requests

|

Connection Pool (25 connections)

conn-01 ————— Postgres

conn-02 ————— Postgres

...

conn-25 ————— Postgres

Requests queue at the pool. Connections are checked out and returned.

Postgres sees 25 connections, not 1,000.

Pool sizing (HikariCP / general recommendation):

```
pool_size = (core_count × 2) + effective_spindle_count
```

4-core Postgres server with SSD:

```
pool_size = (4 × 2) + 1 = 9 per application
```

Rule of thumb for web apps:

```
pool_size ≈ (max_concurrent_requests × avg_query_time_ms) / 1,000
```

Example: 100 concurrent requests, each holds connection for 50ms:

```
pool_size = (100 × 50) / 1,000 = 5 connections needed
```

In-process pool vs PgBouncer:

	In-process pool	PgBouncer
Where	Inside the application process (SQLAlchemy, HikariCP)	External proxy process
Per-process connections	Yes — 10 app servers × 25 = 250 to Postgres	All app servers share → Postgres sees only N
Best for	Single-process apps or when total connection count is within Postgres limits	Many app processes; microservices; serverless

PgBouncer pooling modes:

Mode	Connection held for	Multiplexing	Gotchas
Session	Entire client session	Low	One client = one Postgres connection
Transaction	Duration of a transaction	High	Best for web apps; breaks <code>SET</code> that persists between transactions
Statement	One statement	Highest	Breaks multi-statement transactions

Multi-tenant + PgBouncer critical note: Postgres RLS uses `SET LOCAL app.tenant_id = ?`. `SET LOCAL` is transaction-scoped — it resets when the transaction ends. **Always use PgBouncer in transaction mode with RLS.** Session mode would let the tenant context persist to the next user on the same connection — a serious data-isolation bug.

Pool exhaustion — the failure mode:

Symptom: requests hang waiting for a connection; Postgres CPU is low; the bottleneck is the pool queue.

Diagnosis: `SELECT count(*) FROM pg_stat_activity` — if near `max_connections`, you're at the Postgres ceiling. If well below and still hanging, the application pool is exhausted.

Fix: increase pool size (up to Postgres `max_connections`), reduce query duration, add read replicas for reads, or add PgBouncer to multiplex.

Interview move: "The first thing I check when a Postgres app slows under load is whether it's hitting the connection limit. Pool exhaustion looks exactly like a database performance problem —

high latency, timeouts — but the DB is actually idle. Fix: PgBouncer in transaction mode between the app and Postgres, pool sized to $2 \times$ CPU cores. With RLS for multi-tenancy, transaction mode is mandatory — session mode leaks tenant context across connections."

SQL vs NoSQL

Question	SQL wins	NoSQL wins
Need ACID transactions?	✓	✗
Complex joins and aggregations?	✓	✗
Flexible / evolving schema?	✗	✓
Horizontal write scaling required?	✗	✓
Simple key lookups dominate?	Either	✓
Strict consistency required?	✓	Depends
High write throughput ($> 10K/s$)?	✗ (without sharding)	✓

Default: Start with Postgres. Add NoSQL only when you've hit a specific ceiling Postgres can't solve.

Cache Strategy Selection

Situation	Use
Read-heavy, data changes infrequently, staleness ok	Cache-aside + TTL
Write-heavy, reads must be fresh immediately	Write-through
Extreme write throughput, can tolerate some loss	Write-behind
Access pattern unknown at write time	Cache-aside (lazy)
Static assets with long TTL	Content-hash in URL + max-age

Messaging Pattern Selection

Situation	Use
Background job, one consumer	Queue (SQS Standard)
Background job, must not duplicate, ordering matters	Queue (SQS FIFO)
Fan-out to multiple subscribers	Pub/Sub (SNS)
Audit log, replayable events, multi-consumer	Kafka / Kinesis
Reliable event publish from within a DB transaction	Outbox pattern

Real-Time Delivery Selection

Situation	Use
Result in DB after async process	Client polling
Infrequent server-initiated events, HTTP only	Long polling
Dashboard / live feed, server pushes	SSE
Chat, collaborative editing, gaming	WebSocket
Large-scale durable event stream	Kafka

Consistency Level Selection

Data	Needed	Use
Bank balance, inventory	Linearisable	Postgres strong reads from primary
User sees their own post	Read-your-own-writes	Sticky session to primary, or write-through cache
Social feed	Eventual	Read replicas, Cassandra, cache
Comment thread order	Causal	Causal sessions
Analytics	Eventual	Pre-aggregated, stream processing

REST vs gRPC vs GraphQL

Situation	Use
Public API, browser clients	REST
Internal service-to-service, performance critical	gRPC
Mobile clients needing flexible queries	GraphQL
Streaming RPC (server push)	gRPC server streaming
Simple CRUD	REST

Part 4: Software Engineering Foundations

When this comes up: Senior engineering interviews frequently ask "Walk me through SOLID principles" or "What design patterns have you used?" These questions test whether you've written production code that evolved over time — not just code that worked once. Anchor every principle to a concrete example.

SOLID Principles

Five principles for writing code that's easy to extend, test, and reason about.

S — Single Responsibility Principle

One class, one reason to change.

A class that authenticates users, sends emails, and writes audit logs has three reasons to change. When the email provider changes, you shouldn't touch the authentication code. Split them: one class authenticates, one sends emails, one writes audit logs.

Smell: A class name with "And" in it — `AuthAndEmailHandler`. A class with 500+ lines because every new feature appended a method.

Your example (Kalaam interpreter): The 5-phase pipeline — clean, scan, tokenize, interpret, output — is each a separate function. The cleaner doesn't tokenize. The tokenizer doesn't interpret. A change to tokenization doesn't touch interpretation.

O — Open/Closed Principle

Open for extension, closed for modification.

Add new behaviour by adding new code, not by editing existing, tested code.

Classic mechanism: Define an abstraction (interface, base class). New behaviour = new implementation of the abstraction. The caller depends on the abstraction and never changes.

Your example (Trade Compliance agents): The Researcher agent and Writer agent share the same tool-calling interface. Adding a Validator agent means writing a new class — the orchestrator that calls them doesn't change. The system is open for new agents, closed for modification of the orchestrator.

Smell: Every time you add a new type, you open a `switch` or `if/elif` chain and add a case. That chain is never closed.

L — Liskov Substitution Principle

Subtypes must be substitutable for their base types without breaking the program.

If `S` extends `T`, anywhere a `T` is used, you can substitute an `S` and the program still behaves correctly. A subclass can't promise less than its parent.

Classic violation:

```
class Rectangle:
    def set_width(self, w): self.width = w
    def set_height(self, h): self.height = h

class Square(Rectangle):          # LSP violation
    def set_width(self, w):
        self.width = self.height = w  # breaks Rectangle's independent-axis contract
```

Code that calls `set_width` and `set_height` independently breaks when given a `Square`.

Interview move: "LSP is why I'm skeptical of inheritance for code reuse. I use it for true substitutability — a `PostgresRepository` and an `InMemoryRepository` are both valid `Repository` implementations. But I won't extend a class just to reuse methods if the subtype doesn't honour the parent's full contract."

I — Interface Segregation Principle

Clients should not be forced to depend on interfaces they don't use.

One fat interface with 10 methods → split into 3 focused interfaces. Each caller imports only the interface with the methods it actually needs. Fewer dependencies, easier to mock in tests.

Your example (stringy-core): Each of the 9 modules is a focused group. The caller who needs `maskEmail` imports only the masking module. Named ESM exports let the bundler tree-shake to exactly the functions used — no interface pollution.

D — Dependency Inversion Principle

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Business logic should not import `PostgresSubmissionRepository` directly. It imports a `SubmissionRepository` interface. The wiring (which concrete class to inject) happens at startup.

```
# High-level (domain) — depends on abstraction:
class SubmissionService:
    def __init__(self, repo: SubmissionRepository):
        self.repo = repo

# Low-level (infrastructure) — implements abstraction:
class PostgresSubmissionRepository(SubmissionRepository): ...
class InMemorySubmissionRepository(SubmissionRepository): ... # for tests

# Startup / DI container:
service = SubmissionService(repo=PostgresSubmissionRepository(db))
```


Why it matters: Swap the DB implementation for a test double without touching the service. This is the mechanism that makes hexagonal architecture work.

GoF Software Design Patterns

The 23 Gang of Four patterns grouped by intent — with concrete examples from your projects.

Creational Patterns — How objects are created

Factory Method — Define an interface for creating an object; let subclasses or a factory function decide which class to instantiate.

```
def create_agent(task_type: str) -> Agent:
    if task_type == "open_ended":
        return HermesAgent(soul=SOUL_MD)      # ReAct loop, decides its own path
    elif task_type == "structured":
        return LangGraphWorkflow(graph=QUIZ_GRAPH) # known steps, resumable
```

Your example (01.dev two-engine design): Open-ended tutoring → Hermes. Structured quiz → LangGraph. The caller asks for an agent by task type and doesn't know which engine it gets. Adding a new engine = add a new branch, no caller changes.

Builder — Construct a complex object step by step; separate construction from representation.

Used when an object requires many optional parameters. Avoids telescoping constructors.

```
agent = AgentBuilder() \
    .with_soul("SOUL.md") \
    .with_memory(HindSightMemory(student_id)) \
    .with_tools([search_course_content, generate_notes]) \
    .with_model("qwen2.5:3b") \
    .build()
```

Singleton — Ensure a class has only one instance and provide a global access point.

Appropriate for: DB connection pool, application config, logger. Often overused as a global variable in disguise.

Prototype — Create new objects by cloning an existing one. Useful when object creation is expensive and new objects differ only slightly from a base.

Structural Patterns — How objects are composed

Decorator — Add behaviour to an object dynamically without changing its class.

```
class LoggedRepository(SubmissionRepository):
    def __init__(self, inner: SubmissionRepository):
        self.inner = inner

    def save(self, submission):
        log.info(f"Saving {submission.id}")
        result = self.inner.save(submission)
        log.info(f"Saved {submission.id}")
        return result

# Wrap any repo:
repo = LoggedRepository(PostgresSubmissionRepository(db))
```

Your example (Django middleware): Each middleware wraps the next — authentication wraps the view, rate limiting wraps authentication. Request flows through the chain. Each middleware adds behaviour without modifying the others.

Adapter — Make an incompatible interface work with another. Translates one interface into the one the client expects.

```
class MarathiKeywordAdapter(LanguageAdapter):
    def normalize(self, keyword: str) -> str:
        return MARATHI_TO_TOKEN[keyword]

class HindiKeywordAdapter(LanguageAdapter):
    def normalize(self, keyword: str) -> str:
        return HINDI_TO_TOKEN[keyword]
```

Your example (Kalaam keyword substitution): The interpreter works on normalized tokens. Each language is an adapter that converts native keywords into tokens the interpreter understands. Adding a new language = write one new adapter, no interpreter changes.

Facade — Provide a simplified interface to a complex subsystem.

```
// stringy-core — the _s namespace is a Facade:
import { _s } from 'stringy-core'

_s.maskEmail('user@example.com') // hides Intl API, regex, character manipulation
_s.formatCurrency(1500, 'INR')    // hides locale, Intl.NumberFormat complexity
```

Your example (stringy-core _s namespace): 50+ functions across 9 modules. The _s facade gives one simple import. Internally each function uses Intl API, regex, string operations — the caller sees none of it.

Proxy — Control access to an object. Types: Virtual proxy (lazy init), Protection proxy (auth check), Remote proxy (network call looks local).

Your example (o1.dev grounding gate): Before the LLM is called, the proxy checks cosine similarity ≥ 0.45 . Below threshold: the model is never called, a decline is returned. The chat handler doesn't implement this check — the proxy does. The model is the real subject; the grounding gate is the protection proxy.

Composite — Compose objects into tree structures and treat individual objects and compositions uniformly. Classic example: a file system where `File` and `Directory` both implement the same `FileSystemNode` interface.

Behavioral Patterns — How objects communicate

Observer — When one object changes state, all registered dependents are notified automatically.

```
Exam.is_active transitions True → False
| (Django pre_save signal)
├─ PlagiarismOrchestrator.on_exam_close() → Phase 1: behavioural scoring
└─ AuditLogger.on_exam_close()           → write audit event
```

Your example (CodeMas plagiarism trigger): Django's `pre_save` signal is the Observer pattern. `Exam` is the subject. `PlagiarismOrchestrator` is the observer. When the exam closes, the orchestrator is notified without `Exam` knowing anything about plagiarism.

Strategy — Define a family of algorithms, encapsulate each, and make them interchangeable.

```
class SimilarityStrategy(ABC):
    def score(self, a: str, b: str) -> float: ...

class TfidfCosine(SimilarityStrategy):
    def score(self, a, b): ...      # used in CodeMas plagiarism Phase 2

class FuzzyTokenSetRatio(SimilarityStrategy):
    def score(self, a, b): ...      # used in Munshi vendor reconciliation
```

Your example (Munshi vendor matching): The reconciliation engine holds a `SimilarityStrategy`. Swap in Levenshtein distance or embedding cosine similarity without touching the engine. The algorithm is the variable; the engine is the constant.

Command — Encapsulate a request as an object. Enables queuing, logging, and undo.

```
{
  "type": "execute_submission",
  "submission_id": "uuid",
  "student_code": "...",
  "language": "python",
  "test_cases": [...]
}
```

Your example (CodeMas SQS message): Each SQS message is a Command. It encapsulates the full request. Lambda receives it, executes it. Idempotent (SQS deduplication ID = submission UUID). Retryable (Lambda retries on failure). Can be queued (SQS FIFO).

Template Method — Define the skeleton of an algorithm in a base class; let subclasses override specific steps.

```
class PlagiarismAnalyzer(ABC):
    def analyze(self, exam_id):          # template – fixed skeleton
        submissions = self.load(exam_id)
        flags = self.score(submissions) # ← subclasses override this
        self.save(flags)

    @abstractmethod
    def score(self, submissions): ...

class BehavioralAnalyzer(PlagiarismAnalyzer):
    def score(self, submissions): ...    # Phase 1: paste ratio, speed, tabs

class SimilarityAnalyzer(PlagiarismAnalyzer):
    def score(self, submissions): ...    # Phase 2: TF-IDF + cosine
```

Your example (CodeMas two-phase plagiarism): The orchestration skeleton (load → score → save) is fixed. Phase 1 and Phase 2 override only the scoring step.

State — Allow an object to change its behaviour when its internal state changes. The object appears to change its class.

Your example (o1.dev QuizMe LangGraph): The workflow StateGraph transitions between states: `plan_objectives` → `interrupt` → `generate_quiz` → `interrupt` → `summarize`. The system behaves completely differently in each state — waiting for human input, generating content, summarizing results — driven by which state node is active.

Iterator — Provide a way to access elements of a collection sequentially without exposing the underlying representation.

Your example (Kalaam token walk): The interpreter iterates through `tokens[]` without knowing how they were produced. The tokenizer and interpreter are decoupled through sequential access. The interpreter doesn't know if tokens came from a character scan, a cache, or a file.

Pattern Selection Quick Reference

Problem	Pattern
Need exactly one global instance	Singleton
Complex object with many optional parts	Builder
Object creation varies by type at runtime	Factory Method
Add behaviour to an object without subclassing	Decorator
Make two incompatible interfaces work together	Adapter
Simplify a complex subsystem for callers	Facade
Notify multiple objects of a state change	Observer
Swap algorithms at runtime without caller changes	Strategy
Encapsulate a request as an object for queuing/retry	Command
Fixed algorithm structure, variable steps	Template Method
Control or mediate access to another object	Proxy
Object behaviour changes based on internal state	State
Treat individual objects and groups uniformly	Composite

Part 5: Estimation Worked Examples

Example 1: Design WhatsApp

Assumptions: 2B users, 50% DAU = 1B DAU. 40 messages/day/user. Average message = 100 bytes. 10% include media (avg 100 KB).

QPS:

- Write: $1B \times 40 \div 86,400 = \sim 500K \text{ messages/sec}$
- Read: $\sim 5 \times \text{writes} = \sim 2.5M \text{ reads/sec}$

Storage:

- Text: $500K/\text{sec} \times 100B \times 86,400 \times 365 = \sim 1.5 \text{ PB/year}$
- Media: $50K/\text{sec} \times 100KB \times 86,400 \times 365 = \sim 150 \text{ PB/year}$

Design decisions this math drives:

- 500K write QPS → Cassandra (designed for this; partition by conversation_id)
 - 150 PB/year media → S3 + CDN; thumbnail on upload, lazy-load full
 - 2.5M read QPS → Redis cache for recent messages per conversation
-

Example 2: Design a URL Shortener

Assumptions: 100M new URLs/day. Read:Write = 100:1. 5-year retention.

QPS:

- Write: $100M \div 86,400 = \sim 1,200$ writes/sec
- Read: $\sim 120,000$ reads/sec

Storage:

- Row size: ~ 300 bytes (7-char code + 200-char URL + metadata)
- $100M/day \times 365 \times 5 \text{ years} = 182B \text{ rows} \rightarrow \sim 55 \text{ TB}$

Design decisions:

- 1,200 write QPS → Postgres handles this comfortably
 - 120K read QPS → Redis cache essential (80%+ hit ratio for popular URLs)
 - 55 TB at 5 years → Sharding by short_code hash
-

Example 3: Design Twitter

Assumptions: 300M DAU. 500K tweets/day (1% of users tweet). Read:Write = 1000:1. Average tweet = 280 bytes.

QPS:

- Write: $500K \div 86,400 = \sim 6$ writes/sec (low — Twitter is extremely read-heavy)
- Read: $\sim 6,000$ reads/sec (p50); peak up to **500K reads/sec** (viral content)

Storage:

- Tweets: $500K/day \times 280B \times 365 \times 10 \text{ years} = \sim 500 \text{ GB/year}$ (tiny)
- Media: the real storage problem — $50K \text{ photo tweets/day} \times 200KB = 10 \text{ GB/day}$

Design decision this math drives:

- Storage of text is cheap — the architecture problem is *fan-out* (distributing a tweet to 100M followers)
 - This drives the push vs pull debate in feed generation
-

Part 6: Classic Problem Skeletons

Each problem: Clarify → Estimate → Core Design → Deep Dives to offer

1. URL Shortener

Clarify: Read:write ratio? Custom aliases? Analytics? Expiry?

Core design:

- Short code: `hash(long_url)` → take 7 chars. Collision → retry with suffix.
- DB: Postgres. Schema: `short_code (PK), long_url, created_at, user_id, expiry`
- Cache: Redis. Key: `short_code` → `long_url`. TTL: 24h.
- Redirect: 302 (every hit tracked for analytics) vs 301 (browser caches, no tracking)
- At scale: shard by `short_code` (consistent hashing)

Deep dives to offer: Hash collision handling. Custom alias uniqueness. Analytics pipeline (ClickHouse for click events). Multi-region with geo-routing.

2. Rate Limiter

Clarify: Per user? Per IP? Per API key? What happens on exceed — reject or queue?

Core design:

- Algorithm: Token bucket (burst-tolerant, steady refill)
- Storage: Redis. Key: `rate_limit:{user_id}:{window}`. Lua script for atomic check-and-decrement.
- Placement: API Gateway layer before traffic reaches services
- Response: HTTP 429 with `Retry-After` header

Deep dives to offer: Sliding window vs fixed window. Distributed rate limiting across multiple gateway nodes (Redis shared state). Soft limits (warn) vs hard limits (reject). Priority tiers.

3. Notification Service

Clarify: Channels (push, email, SMS, in-app)? Volume? Delivery guarantees? User preferences?

Core design:

- Event producer → Kafka (one topic per channel type)
- Per-channel workers consume independently: push worker, email worker, SMS worker
- Deduplication: `notification_id` as idempotency key (Redis SET NX)
- User preferences: DB table — per user, per notification type, per channel (opt-out)

Deep dives to offer: Thundering herd on bulk notifications (rate-limit workers). Priority lanes (transactional vs marketing). Retry on failed delivery. Delivery receipts. Unsubscribe handling.

4. Chat System

Clarify: 1-to-1 or group? Max group size? Persistence? Online presence?

Core design:

- WebSocket for real-time delivery. Each chat server holds connections for a set of users.
- Message path: sender → chat server → Kafka → receiver's chat server → WebSocket push.
- Chat server registry in Redis: `user_id` → `server_id` (so the sender's server knows where to route).
- Storage: Cassandra. Partition key: `conversation_id`. Cluster key: `message_timestamp` (DESC for

recent-first reads).

- Offline delivery: when receiver's chat server doesn't find an active WebSocket, writes to their message inbox in DB. Client fetches on next connect.

Deep dives to offer: Message ordering. Group fan-out at scale (fan-out on write for small groups; fan-out on read for large groups). End-to-end encryption. Message deletion / unsend. Read receipts.

5. Twitter / News Feed

Clarify: Post + read volume? Celebrity accounts? Real-time vs eventual? Personalisation?

Core design — hybrid fan-out:

- Normal users (< 1M followers): **fan-out on write** — when they post, push to all followers' feed caches in Redis (sorted set by timestamp). Instant reads.
- Celebrities (> 1M followers): **fan-out on read** — skip pre-populating. At read time, merge cached feed + recent celebrity tweets.
- Why hybrid: fan-out on write for 1M followers means 1M Redis writes per tweet — too expensive for celebrities.

Storage: Cassandra for tweets. Redis sorted sets for per-user feed caches (keep last 800 tweets max).

Deep dives to offer: Trending topics (stream processing, sliding window count). Search (Elasticsearch). Recommendation engine (collaborative filtering). Ad targeting. Pagination.

6. Distributed Job Scheduler

Clarify: One-time vs recurring? Delay SLA? Priority? Exactly-once?

Core design:

- Jobs table in Postgres: `id, run_at, status (pending/running/done/failed), payload, retry_count`
- Scheduler: `SELECT FOR UPDATE SKIP LOCKED WHERE run_at <= NOW() AND status = 'pending' LIMIT 10` — workers compete without conflicts
- Workers: process job, update status to done or failed, compute next_run_at for recurring
- Missed jobs (system down during run_at): scheduler catches up on start

Deep dives to offer: Exactly-once execution (idempotency in the job itself). Priority queues (multiple tables or priority column with index). Fan-out jobs (1 parent → N child jobs). Distributed workers (partition by queue name). Dead jobs (max_retries exceeded → DLQ table).

7. Key-Value Store (Design Redis)

Clarify: Persistence? Max value size? Replication? Consistency level?

Core design:

- In-memory hash map for O(1) get/set
- WAL (write-ahead log) for crash recovery
- Consistent hashing to distribute keys across nodes

- Replication: write to N nodes; quorum $W + R > N$ for strong consistency
- Compaction: SSTable + LSM tree for write-optimised disk layout

Deep dives to offer: LSM tree vs B-tree. Conflict resolution (vector clocks, last-write-wins). Gossip protocol for cluster membership. Failure detection. Leader election per shard.

8. Video Streaming (Design YouTube)

Clarify: Upload? Playback? Recommendations? Comments? Scale?

Core design:

- **Upload:** Client → Upload service → raw video to S3 → enqueue transcoding job → Fargate/Lambda transcodes to multiple resolutions (360p/720p/1080p/4K) → store segments in S3
- **Playback:** Client → CDN (serves HLS/DASH segments directly from S3). Origin is never hit during playback. Adaptive bitrate: client switches quality based on bandwidth.
- **Metadata:** Postgres for video metadata. Cassandra for view counts (write-heavy increment).
- **Never serve video from the origin server directly.**

Deep dives to offer: Transcoding pipeline (ffmpeg, parallelism). Adaptive bitrate (HLS segments + .m3u8 manifest). CDN cache invalidation on re-upload. Recommendation system (matrix factorisation or two-tower neural net). Anti-abuse (CSAM, copyright detection).

9. Web Crawler

Clarify: Domain scope (entire web vs. specific sites)? Freshness (re-crawl frequency)? Scale (pages/sec)? Robots.txt respect? Deduplication required?

Core design:

- **Seed URLs** → URL Frontier (priority queue, ordered by page rank + recency)
- **Crawlers** (stateless workers) dequeue URLs, fetch with HTTP, extract links
- **DNS caching** at the crawler — avoid per-URL DNS lookups (expensive at scale)
- **Deduplication** — URL seen set (Redis Bloom filter or hash set) + content hash to detect duplicate pages at different URLs
- **Storage** — raw HTML → S3; metadata (URL, crawl_time, status, outlinks) → Postgres or Cassandra



Politeness: Respect `robots.txt` — fetch and cache per domain on first visit. Rate-limit per domain (1 req/sec default). Use `crawl-delay` directive if present.

Re-crawl strategy: High-value pages (news homepages) re-crawled every hour. Static docs re-crawled weekly. Detect change frequency from response headers (`Last-Modified` , `ETag`).

Scale math example: 1B pages, 10KB each → 10TB HTML storage. 10 pages/sec/worker → need 1,157 workers to crawl 1B pages in 24 hours.

Deep dives to offer: URL normalisation (canonical URL dedup). JavaScript rendering (headless Chrome for SPAs). Distributed URL frontier (Kafka topics by domain). Anti-bot evasion (User-Agent, rate limiting). Content dedup via SimHash.

Part 7: Your Projects as Proof Points

When the interviewer asks about a pattern, here's your lived evidence:

Pattern / Concept	Your Project	One-line proof
Message queue + async execution	CodeMas	SQS FIFO → Lambda → Postgres; client polls result
Exactly-once delivery (deduplication)	CodeMas	SQS FIFO dedup on submission UUID; 5-min window
Row-level lock / SELECT FOR UPDATE	CodeMas	Attempt counter lock prevents double-count race
Client polling (stateless result delivery)	CodeMas	Frontend polls GET /result/{id}/ every 1.5s
Batch post-processing pipeline	CodeMas	Plagiarism triggered by Exam.is_active pre_save signal
TF-IDF + cosine similarity at scale	CodeMas	Plagiarism Phase 2; $O(K \times N)$ not $O(N^2)$ via behavioral pre-filter
Behavioral anomaly scoring	CodeMas	4-signal weighted score: paste ratio + speed + tabs + attempts
DLQ / failure isolation	CodeMas	SQS DLQ for Lambda failures; exam unaffected
RAG (Retrieval-Augmented Generation)	the01.dev	Course content retrieval with 0.45 grounding threshold
Vector search (HNSW / ANN)	the01.dev	pgvector ANN index for semantic retrieval at query time
Grounding threshold (deterministic refusal)	the01.dev	Below 0.45 cosine similarity: model never called
Agent runtime (ReAct loop)	the01.dev / Munshi / Trade	Hermes: reason → tool call → observe → repeat
HITL (Human-in-the-Loop)	the01.dev (QuizMe)	LangGraph interrupt: human approves before quiz generates
Deterministic workflow engine	the01.dev	LangGraph StateGraph with MemorySaver checkpointer
Self-improving AI system (eval → propose → gate)	the01.dev	GEPA: eval → reflect → propose → Pareto gate → human approves
Evaluation framework	the01.dev	RAGAS: faithfulness + context precision, sovereign judge
Local model serving (sovereign inference)	the01.dev / Munshi / Trade	Ollama/vLLM; zero external generation API calls
EU-AI-Act audit logging	the01.dev	Row written before every inference: who, what, model, SOUL hash
MCP (typed tool protocol)	the01.dev / Munshi / Trade	FastMCP stdio server; arguments schema-validated before execution
Financial decimal arithmetic	Munshi	Python Decimal for GST at ₹5Cr scale; float gives wrong answers

Pattern / Concept	Your Project	One-line proof
Fuzzy string matching	Munshi	Token set ratio for vendor name reconciliation vs GSTR-2A
Multi-language interpreter design	Kalaam	5-phase pipeline: clean → scan → tokenize → interpret → output
Offline-first PWA	Kalaam	Service worker + cache-first; zero API calls at runtime
npm library with tree-shaking	stringy-core	ESM named exports, zero deps, Babel for Jest compat
Multi-agent orchestration	Trade Compliance	Researcher agent → Writer agent; independent SOUL.md per agent

Part 8: Technical Dictionary

Distributed Systems

CAP Theorem — A distributed system can guarantee at most two of: Consistency (all nodes return the same data), Availability (every request gets a response), Partition Tolerance (works despite network failures). Since partitions are unavoidable, the real choice is CP (return an error during a partition) vs AP (return potentially stale data).

PACELC — Extension of CAP. Even when there is No partition, you still trade Latency vs Consistency. PACELC forces you to think about the everyday trade-off, not just the rare partition scenario. More relevant for most product decisions than CAP alone.

Eventual consistency — Given enough time with no new writes, all replicas will converge to the same value. You may read stale data briefly. Example: your follower count on Twitter might be 1 second behind the true value.

Linearisability (strong consistency) — The strongest guarantee. Every operation appears to take effect atomically at a single point between its start and end. Every read returns the latest completed write from any client. Example: bank balance — you must never see a stale value.

Consensus — Getting a distributed cluster to agree on a single value. Algorithms: Raft (readable, used in etcd), Paxos (original, complex). Used in: leader election, distributed locks, replicated state machines.

Split brain — Two partitions each believe they're the active primary and both accept writes. When the partition heals, you have conflicting data with no ground truth. Prevention: quorum writes (require majority acknowledgement before committing).

Quorum — A majority of nodes ($N/2 + 1$). For a 5-node cluster, quorum = 3. Operations requiring quorum can tolerate ($N - \text{quorum}$) node failures while maintaining consistency.

Storage & Databases

ACID — Atomicity (all or nothing), Consistency (constraints always satisfied), Isolation (concurrent transactions don't interfere), Durability (committed data survives crashes). The guarantee of relational databases.

BASE — Basically Available, Soft state, Eventually consistent. The NoSQL trade-off: sacrifices strict consistency for availability and horizontal scalability.

LSM Tree (Log-Structured Merge Tree) — Write-optimised storage. All writes go to an in-memory buffer (MemTable), flushed to sorted immutable files (SSTables) on disk. Reads merge multiple SSTables. Used by: Cassandra, RocksDB. Fast writes, slightly slower reads than B-tree.

B-Tree — Read-optimised storage. Data in sorted tree pages on disk. Reads: $O(\log n)$ page fetches. Writes update in place. Used by: Postgres, MySQL. Faster reads, slower sequential writes vs LSM.

WAL (Write-Ahead Log) — Before modifying data on disk, the change is written to an append-only log first. If the system crashes mid-write, the WAL is replayed to restore consistency. Also used for CDC (reading the log to capture change events without touching the application).

Consistent hashing — A hashing scheme where adding or removing a node only remaps $\sim 1/N$ of keys (not all of them). Used in distributed caches and databases to minimise data migration when nodes join or leave.

JSONB — Postgres's binary JSON column type. Stored as binary (not text) so it's faster to query than the `json` type. Supports GIN indexes for querying nested fields. Useful for flexible schema within a relational DB.

Connection pool — A cache of database connections reused across requests. Opening a new Postgres connection costs ~ 5 ms and a backend process. A pool of 20 connections serves hundreds of concurrent requests by reusing them. PgBouncer is a Postgres-specific connection pooler.

Caching

Cache hit ratio — The percentage of reads served from cache without hitting the database. 80% hit ratio means only 20% of reads reach the DB. A cache is only worth its operational cost if the hit ratio is high enough to meaningfully reduce DB pressure.

Eviction policy — The rule for removing items when the cache is full. LRU (Least Recently Used): remove what hasn't been accessed longest. LFU (Least Frequently Used): remove least-accessed. TTL: remove when timer expires.

Cache-aside (lazy loading) — The application manages the cache. On miss: app reads from DB, populates cache, returns. On write: app writes to DB and invalidates (or updates) cache. Most common pattern; app has full control.

Write-through — Every write goes to both cache and DB. Cache always up to date. Writes are slightly slower. Reads always fast.

Write-behind (write-back) — Writes go to cache only. An async process flushes to DB. Extremely fast writes. Risk: data in cache-only state is lost if the cache crashes before the flush.

TTL (Time to Live) — How long a cached item remains valid. After it expires, the next read is a cache miss and fetches fresh data from the source.

Cache stampede — A popular cache key expires. Thousands of concurrent requests all miss at the same instant, all hit the database simultaneously. Mitigation: probabilistic early refresh (renew slightly before TTL expires) or fetch-locking (one thread fetches; others wait for it).

Messaging

Idempotency — An operation that produces the same result no matter how many times it executes. Example: `SET balance = 1000` is idempotent; `INCREMENT balance BY 100` is not. Idempotent consumers can safely process duplicate messages without side effects.

Dead Letter Queue (DLQ) — A holding area for messages that repeatedly fail processing. After N failed attempts, the message moves to the DLQ instead of blocking the main queue. Allows manual inspection and replay.

Partition key (Kafka) — Determines which partition a message is routed to. Messages with the same partition key always go to the same partition, guaranteeing ordering within that key. Example: all events for `order_id=123` on the same partition.

Consumer group (Kafka) — A set of consumers that collectively consume a topic. Each partition is read by exactly one consumer in the group. Multiple independent consumer groups can each read the full topic independently (no interference).

Offset (Kafka) — The sequential position of a message within a partition. Consumers commit their current offset. On restart, they resume from the last committed offset — enables exactly-once-style processing and replay.

API & Web

Idempotency key — A client-generated unique ID sent with a POST request (e.g., payment). The server stores it; if the same key arrives again (network retry), the server returns the stored result without re-executing. Safe retries on non-idempotent operations.

Cursor pagination — "Next page" is identified by a cursor (last item's ID or timestamp), not an offset. Immune to insertions mid-pagination. Efficient on large tables — the DB can seek to the cursor position via index, not scan to offset.

Webhook — A reverse API call. Instead of polling an external service, you register a URL and the service POSTs to you when an event happens. Example: Razorpay POSTs to `/webhook/payment` when payment succeeds.

Service mesh — Infrastructure layer handling service-to-service communication (mTLS, retries, circuit breaking, observability) without application code changes. Implemented as sidecar proxies (Envoy). Examples: Istio, Linkerd.

AI & ML Systems

RAG (Retrieval-Augmented Generation) — Retrieve relevant documents from a knowledge base first, then pass them as context before the model generates a response. Grounds the model in real content; dramatically reduces hallucination compared to generation from training data alone.

Embedding — A dense vector (list of numbers) representing the meaning of text. Sentences with similar meaning produce vectors that are numerically close together in vector space. Computed by an embedding model (OpenAI text-embedding-3-small, sentence-transformers, etc.).

HNSW (Hierarchical Navigable Small World) — A graph-based algorithm for fast approximate nearest-neighbor search. Builds a multi-layer graph connecting similar vectors. Search traverses from the top layer (coarse) to the bottom (fine), visiting only a small fraction of vectors. Finds nearest neighbors in milliseconds across millions of vectors.

ANN (Approximate Nearest Neighbor) — Finding vectors "close enough" to a query without checking every vector in the database. Trades a small accuracy loss for massive speed gain. Acceptable for RAG because a slightly imprecise match is almost always still relevant context.

Grounding threshold — A minimum similarity score below which the system refuses to call the model at all — enforced in application code, not as a model instruction. Deterministically prevents hallucination by ensuring the model is only called when relevant context exists.

Agent (AI) — A model in a loop that calls tools, reads results, and decides its next action autonomously until it reaches a conclusion. Unlike single prompt-response, an agent can do multiple rounds of reasoning and tool use.

ReAct loop (Reasoning + Acting) — The agent generates a thought (what to do), takes an action (tool call), observes the result, reasons again. Continues until the model decides it has sufficient information to answer.

MCP (Model Context Protocol) — A typed, schema-validated interface for exposing tools to AI models. Tool arguments are validated against a JSON schema before execution. Invalid arguments return an error the model can read and correct — unlike untyped function calls.

SOUL.md — A markdown file loaded as the system prompt that defines an agent's identity, personality, constraints, and hard rules. Separating identity from code means agent behaviour can change without a code deployment.

Sovereign inference — Running AI generation entirely on infrastructure you control, with no external API calls for the generation step. Guarantees data never leaves your servers. Implemented with local model servers (Ollama for dev, vLLM for production).

GEPA (Generative Evaluation and Prompt Advancement) — A self-improvement pipeline for agent identity (SOUL.md). Steps: eval current SOUL on a fixed test set → reflect on weakest cases → propose an improved SOUL → gate with Pareto criterion → require human approval → write with backup. Improves agent quality without touching code.

Pareto gate — A quality filter accepting a new system version only if it improves the average score AND no individual test case scores below a floor. Prevents "better on average but regressed on edge cases" — the most common failure mode of automated improvement loops.

RAGAS — A RAG evaluation framework. Measures faithfulness (are all claims in the model's answer supported by retrieved chunks?) and context precision (were the retrieved chunks actually relevant to the question?). Used to measure and improve RAG pipeline quality.

Algorithms

TF-IDF (Term Frequency-Inverse Document Frequency) — A method to convert text into a vector by weighting words. Words that appear frequently in one document but rarely across all documents get high weight — they are characteristic of that document. Used in search and plagiarism detection.

Cosine similarity — Measures the angle between two vectors. Range: -1 to 1. Score of 1 = identical direction (same meaning). Score of 0 = orthogonal (unrelated). Used to compare TF-IDF vectors for similarity search and plagiarism detection.

Levenshtein distance — The minimum number of single-character edits (insert, delete, replace) to transform one string into another. Computed with an $O(m \times n)$ dynamic programming matrix. Used for fuzzy matching and spell checking.

Consistent hashing — A technique where hash keys are mapped onto a ring. Nodes own arcs of the ring. Adding or removing a node only affects keys that were owned by that node — not all keys. Minimises reshuffling in distributed caches and databases.

Token bucket — A rate limiting algorithm. A bucket holds up to N tokens; each request consumes one; tokens refill at a fixed rate R per second. Allows bursts up to N while enforcing an average rate of R . The most common rate limiting algorithm in production.

LSM tree compaction — SSTables (sorted string table files) accumulate on disk from MemTable flushes. Compaction merges multiple SSTables into one, removes deleted/overwritten entries, and keeps disk usage bounded. The background cost of write-optimised storage.

Architecture & Networking

Forward proxy — A proxy that sits in front of clients. Clients route outbound traffic through it. The destination server sees the proxy's IP, not the client's. Used for content filtering, anonymity, and caching in corporate environments.

Reverse proxy — A proxy that sits in front of servers. Clients call the proxy; it forwards to the appropriate backend. The backend sees the proxy's IP. Handles SSL termination, load balancing, caching, and WAF. Every production web server uses one.

Monolith — A single deployable artifact containing all application functionality. One codebase, one process, one database. Fast to develop early. Harder to scale and deploy independently at large team sizes.

Modular monolith — A monolith internally structured into enforced modules with clear boundaries. Modules interact only via defined APIs. Easier to split into microservices later when boundaries are already clean.

Microservices — An architecture where each business capability is a separate, independently deployable service with its own codebase and database. Enables team autonomy and independent scaling. Adds distributed systems complexity.

Distributed monolith — An anti-pattern: microservices that are so tightly coupled they must deploy together. All the operational cost of microservices with none of the independence. Caused by shared databases, synchronous call chains, or no clear domain ownership.

Event-Driven Architecture (EDA) — Components communicate via events through a broker rather than direct calls. Producer emits an event without knowing who consumes it. Consumers react independently. Enables loose coupling and parallel processing.

BFF (Backend for Frontend) — A separate backend service tailored to a specific client type (mobile, web, third-party). Aggregates microservice calls, shapes responses for the client, handles client-specific auth. Eliminates over-fetching without polluting the core API.

DNS (Domain Name System) — A globally distributed system that translates domain names (api.example.com) to IP addresses. Hierarchical: root servers → TLD servers → authoritative nameservers. Results cached by TTL.

DNS A record — Maps a domain to an IPv4 address directly.

DNS CNAME record — Maps a domain to another domain (alias). The final resolution is the target domain's A record.

TTL (DNS Time To Live) — How long DNS resolvers cache a response before re-querying. Low TTL = fast failover, more DNS traffic. High TTL = less traffic, slower propagation.

Geohashing — Encoding a geographic coordinate (lat, lon) as a short string. Nearby locations share a common string prefix, enabling fast proximity lookups via a single database range scan on the prefix.

Quadtree — A spatial data structure where each node represents a rectangular region with exactly 4 children (NW, NE, SW, SE). Subdivides until each leaf has at most N points. Adapts to density — small cells where data is dense, large cells where sparse.

RTO (Recovery Time Objective) — How long the system can be down before business damage becomes unacceptable. The target for how quickly to restore service after a failure.

RPO (Recovery Point Objective) — How much data loss is acceptable. The target for how old the most recent backup or replica can be at the time of failure.

Pilot light — A DR strategy where only the minimum core of the system runs continuously (usually the database replica). Application servers are off but can be started quickly. Faster than cold backup-restore; cheaper than warm standby.

Warm standby — A DR strategy where a scaled-down replica of full production runs continuously. On failure, scale up. RTO measured in minutes.

Virtual Machine (VM) — A software emulation of a physical computer. Runs a full OS with its own kernel. Strong isolation. GBs of disk, minutes to start. Managed by a hypervisor.

Container — A lightweight process-level isolation using the host OS kernel. Shares kernel with host. MBs of image, seconds to start. Weaker isolation than VM. Docker is the dominant container runtime.

Orchestrated by Kubernetes.

Hypervisor — Software that creates and manages virtual machines. Type 1 (bare-metal): runs directly on hardware. Type 2 (hosted): runs as an app on a host OS.

Database Federation — Splitting a single database into multiple databases by functional domain (users DB, orders DB, products DB). Different from sharding — splits by table function, not by row. Each domain scales independently.

Materialized view — A pre-computed query result stored as a physical table. Unlike a regular view, it is computed once and refreshed on schedule. Eliminates expensive real-time join/aggregation queries at the cost of staleness.

N+1 query problem — A loop that fetches N parent records in one query, then issues N additional queries for related records. Total: N+1 queries. Fix: eager loading (JOIN) or batch loading (WHERE id IN (...)).

Master-Master replication — Both database nodes accept writes. Changes are replicated to each other. Enables high write availability and geographic distribution. Requires conflict resolution when concurrent writes target the same row.

MapReduce — A programming model for parallel batch processing at scale. Map phase: transform each input record into (key, value) pairs, independently and in parallel. Reduce phase: aggregate all values for each key. Framework handles data splitting, scheduling, and fault tolerance. Used in Hadoop. Spark replaced it for most use cases with in-memory processing.

Snowflake ID — A 64-bit globally unique, time-ordered ID. Structure: 41 bits timestamp + 10 bits machine ID + 12 bits sequence. No coordination required (machine ID assigned at startup). Generates up to 4,096 unique IDs per millisecond per machine. Sortable — higher ID = newer entry. Used by Twitter, Discord, Instagram.

Bloom filter — A space-efficient probabilistic data structure for set membership. Returns "definitely not in set" or "probably in set" (small false positive rate, never false negatives). Used for URL deduplication in web crawlers, cache-miss optimization (don't query DB for keys that definitely don't exist).

TCP (Transmission Control Protocol) — Connection-oriented transport protocol. Guarantees delivery, ordering, and flow control. Higher overhead than UDP. Used when all data must arrive: HTTP, databases, file transfer.

UDP (User Datagram Protocol) — Connectionless transport protocol. Best-effort delivery — packets may be dropped or arrive out of order. Lower overhead, lower latency. Used when speed beats guaranteed delivery: video calls, live game state, DNS.

OLTP (Online Transaction Processing) — Databases optimised for high-throughput, low-latency transactional operations — inserts, updates, point lookups. Row-oriented storage. Examples: Postgres, MySQL. The primary database for most web applications.

OLAP (Online Analytical Processing) — Databases optimised for large aggregation queries across many rows (e.g., "total revenue by region last year"). Column-oriented storage — reads only the columns needed, compresses well, fast scans. Examples: ClickHouse, BigQuery, Redshift. Never the primary transactional database.

Refresh-ahead — A cache update strategy where the cache proactively refreshes entries before they expire, based on predicted access patterns. The client always gets a cache hit (no miss latency). Requires the system to know which keys will be requested next. Fails if prediction accuracy is low (useless refreshes waste compute).

Infrastructure & Deployment

Load balancer — A server (or cluster) that distributes incoming requests across a pool of backend servers. Performs health checks and removes unhealthy servers from rotation automatically. Layer 4 operates at TCP level; Layer 7 at HTTP level with content-aware routing.

Sticky session (session affinity) — Routing all requests from a given client to the same backend server. Required when in-process session state exists. Anti-pattern in horizontally scaled systems — prefer external session stores (Redis) so any server can handle any request.

Horizontal scaling — Adding more machines to handle more load. Requires stateless services. Enables high availability through redundancy. The correct response to sustained throughput bottlenecks.

Vertical scaling — Upgrading to a more powerful machine (more CPU, RAM). Simple, no code changes. Has a hard ceiling. Single point of failure. The correct first response to latency bottlenecks.

Service discovery — The mechanism by which services in a cluster find each other's current addresses. DNS-based (Kubernetes Services), client-side (Consul, Eureka), or server-side (load balancer + registry). Necessary because IP addresses change as instances are added or replaced.

Health check — An endpoint or probe that reports whether a service instance is functioning correctly. Liveness (`/healthz`) checks if the process is alive; readiness (`/readyz`) checks if it can currently serve traffic. Used by load balancers and Kubernetes to route traffic only to healthy instances.

Blue-green deployment — Running two identical production environments. Traffic switches from Blue (current) to Green (new) at the load balancer. Instant rollback by switching back. Requires 2× infrastructure during the transition.

Canary deployment — Routing a small percentage (1–5%) of traffic to a new version. Monitoring confirms it's healthy before expanding to 100%. Limits blast radius of a bad deploy to a small fraction of users.

Feature flag — A runtime toggle that enables or disables code paths without a new deployment. Decouples deploy from release. Enables gradual rollouts, A/B testing, and instant kill-switch for problematic features.

Presigned URL — A time-limited URL generated by the server that grants the holder temporary permission to access a specific S3 object directly. The client downloads from S3 without the request passing through the application server.

Multipart upload — Uploading a large file by splitting it into parts and uploading concurrently. Enables parallel upload (faster), resumable transfers (restart only failed parts), and bypasses single-request size limits. Standard for files > 100MB.

Active-Active — All regions/nodes accept writes simultaneously. Highest availability and write throughput. Requires conflict resolution strategy for concurrent writes to the same data.

Active-Passive — One region/node is primary (handles writes). Others are replicas (read-only standby). Simpler consistency model. Failover requires promoting a replica, which has downtime measured in minutes.

Backpressure — A mechanism for a consumer to signal a producer to slow down when it can't keep up. Without it, a fast producer overwhelms a slow consumer. Implemented via bounded queues, consumer autoscaling, producer rate limiting, or load shedding.

Schema registry — A central service that stores and versions message schemas (Avro, Protobuf) for event streams. Enforces compatibility rules before a new schema is published. Prevents producers and consumers from silently breaking each other on schema changes.

Two-phase commit (2PC) — A distributed transaction protocol ensuring all-or-nothing commits across multiple systems. Phase 1: coordinator asks all participants "can you commit?" Phase 2: if all say yes, commit; otherwise abort. Blocking — if the coordinator fails after Phase 1, participants hold locks indefinitely.

Saga — An alternative to 2PC for distributed transactions. A sequence of local transactions where each step publishes an event; failure triggers compensating transactions. Non-blocking and highly available; accepts eventual consistency instead of true atomicity.

Secrets manager — A service for securely storing, rotating, and auditing access to sensitive credentials (DB passwords, API keys, certificates). Avoids hardcoding or storing secrets in environment files. Examples: AWS Secrets Manager, HashiCorp Vault.

Dynamic secrets — Credentials generated on-demand with a short TTL (e.g., a DB username/password created by Vault for one app instance for 1 hour, then revoked). Compromise is limited to one instance and one TTL window.

Composite index — A database index on multiple columns. Used by queries that filter on the leftmost prefix of the indexed columns. `INDEX(user_id, created_at)` speeds up queries filtering on `user_id` or `user_id + created_at`, but not `created_at` alone.

Covering index — An index that contains all columns needed by a query, so the query can be answered from the index without reading table rows. Eliminates I/O for the data rows, giving index-only scan performance.

Partial index — An index that only covers rows matching a WHERE condition. Smaller, faster, cheaper than a full-table index. Ideal for low-selectivity columns where only a subset of rows is ever queried (`WHERE status = 'PENDING'`).

Print this. Read it before every system design interview. The framework in Part 1 tells you how to run the room. The patterns in Part 3 give you the vocabulary. The proof points in Part 7 give you the credibility.